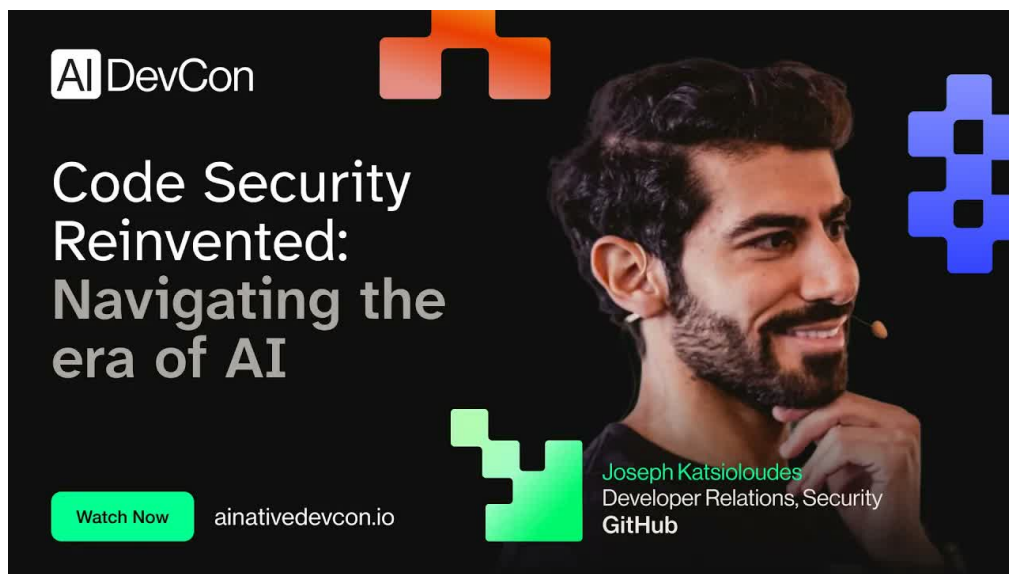


Code Security Reinvented: AI 时代的代码安全重塑

SSS & Codex

2026-06-29



视频作者/频道: AI Native Dev

发布日期: 2026-06-03

视频时长: 35:17

视频标题: Joseph Katsiolouides - Code Security Reinvented: Navigating the era of AI - AI Native DevCon June 26

视频链接: https://www.youtube.com/watch?v=gc5_ICZg9tg

字幕证据: YouTube 人工英文字幕 en-US, 本地文件 source/subtitles/gc5_ICZg9tg.en-US.srt

PDF 制作 Skill: youtube-render-pdf

目录

1 安全缺口为什么适合用 AI 缩小	4
1.1 本节结论：AI 的入口是能力缺口，控制点是人在回路中	4
1.2 GitHub Security Lab 的可信度：它把修复放在报告之后	4
1.3 1:100 的压力点：安全专家无法线性覆盖开发者	5
1.4 Human in the loop：让 AI 提议，让人和系统负责	5
1.5 全局命题：缩小缺口的路径是把安全能力放进工作流	6
1.6 本章小结	6
2 写更安全的代码：幻觉、摇摆与修复问题	7
2.1 本节结论：原始模型输出不能单独承担安全把关	7
2.2 从“shift left”到“start left”：安全必须进入代码生成现场	7
2.3 幻觉 demo：模型说得像真的，也可能说错	8
2.4 非确定性 demo：上下文变小，漏洞反而消失	8
2.5 换模型不能自动消除问题：需要脚手架和检测来源	9
2.6 修复问题：安全价值来自更快到达可验证补丁	10
2.7 安全卫生：AI 不能成为唯一安全网	10
2.8 本章小结	11
3 MCP 与 Skills：能力加流程才可用	11
3.1 本节结论：安全助手需要可信上下文，也需要可审计流程	11
3.2 MCP：把模型从训练语料带到可信工具现场	11
3.3 MCP 的卫生规则：只把信任给已经可信的工具	12
3.4 GitHub MCP：把代码安全、Secret Protection 与 Advisory 带进 agent 上下文	12
3.5 传统静态分析覆盖模式，AI 推理覆盖行为与上下文	13
3.6 Skills：把专家流程写成可维护的 agent 操作说明	14
3.7 能力与流程的分层：MCP 负责能做什么，Skills 负责怎样做	14
3.8 从 finding 到 PR：本章为下一章搭桥	15
3.9 本章小结	16
4 从仪表盘到 PR：把修复放回工作流	16
4.1 本节结论：AI 修复必须进入开发者已经工作的地方	16
4.2 旧工作流：CodeQL 仪表盘告诉你哪里错了，却离修复还有距离	16
4.3 新工作流：Autofix 把解释和候选代码带到 PR	17
4.4 PR 是工作面：减少工具切换就是减少安全摩擦	18
4.5 三倍更快：速度来自链路缩短，仍需验证	19
4.6 人工 Review、CI/CD 与安全验证仍是合入门槛	19

4.7	从本章看全局：AI 安全价值来自修复链路设计	19
4.8	本章小结	20
5	Agentic Workflows 与 Task Flows：把专家知识写进自动化	20
5.1	本节结论：智能体要像安全作业一样运行	20
5.2	为什么 SAST 仍然要留在体系里	21
5.3	仓库里的安全智能体：从脚本规则到触发式运行	21
5.4	指令设计的成本：上下文膨胀会让流程变钝	22
5.5	Task Flows：把安全研究员的审查路线公开出来	22
5.6	风险边界：自动化越聪明，越要留下人和工具的刹车	23
5.7	本章小结	24
6	供应链决策：更快地研究依赖风险	24
6.1	本节结论：AI 的作用是缩短研究时间，风险决定仍由人承担	24
6.2	从三分钟问题开始：依赖引入前到底该研究什么	24
6.3	gh.io/risk：开放 instruction files 与 executive summary	25
6.4	Bootstrap 示例：问清楚“别人会怎样攻击我”	26
6.5	可复用依赖风险清单：从摘要回到证据	26
6.6	Risk appetite：同一个依赖，在不同系统里的答案可能不同	27
6.7	本章小结	27
7	Fuzzing 与 Playground：用练习环境教会安全直觉	27
7.1	本节结论：AI 最适合把失败模式变成可练习的经验	27
7.2	Fuzzing：用大量输入寻找不良行为	28
7.3	隐私与上下文泄漏：训练环境也要解释边界	29
7.4	Playground：两分钟进入沙箱，先在假世界里犯错	29
7.5	自然语言攻击训练：不懂代码的人也能理解泄密风险	30
7.6	安全教育要从被动学习转向动手练习	30
7.7	把本节放回全局：安全能力来自反复练习	30
7.8	本章小结	31
8	Q&A：误报、SLO、LLM Judge 与最小权限	31
8.1	本节结论：Q&A 把演示收束成组织操作模型	31
8.2	误报的真实成本：五个 false positives 可能拖住整个开发队伍	32
8.3	降低误报：多模型、多次运行、静态工具与 AI 推理叠加	33
8.4	供应商责任：开发者不该自己拼装全部安全管线	33
8.5	开发者责任：安全 SLO 把质量要求写进交付目标	33
8.6	LLM-as-judge：第二个模型可以当陪审员	34

8.7 Judge 的局限：攻击者只需要成功一次	35
8.8 最小权限：AI 不该接触的东西，一开始就不能给	35
8.9 本章小结	36
9 总结与延伸	36
9.1 总论：AI 安全的主线是把安全能力送回开发流程	36
9.2 从能力缺口到修复链路：真正短缺的是闭环能力	37
9.3 MCP、Skills 与 Task Flows：让模型按专家路线使用工具	37
9.4 组织责任：安全应进入 SLO、PR 和教育系统	37
9.5 边界控制：最小权限比漂亮提示词更可靠	38
9.6 主持人的结尾：演讲本身也可以变成 skill	38
9.7 实践清单：团队可以从五件事开始	38

1 安全缺口为什么适合用 AI 缩小

1.1 本节结论：AI 的入口是能力缺口，控制点是人在回路中

本章的核心结论很直接：这场演讲把 AI 安全的起点放在一个组织现实上，应用安全专家太少，开发者太多；AI 的合理位置是在开发流程中放大安全专家的知识、提示开发者更早发现风险、加快修复速度，同时保留人的判断、审查和测试。Joseph Katsioloudes 在开场说明，他的目标是展示 “practical ways to use artificial intelligence for security use cases”，并且这些方法可以放到 GitHub Copilot、Claude Code、Codex 等工具中实践（00:01:01–00:01:12）。

全局定位

本章的总论点是：AI 可以缩小安全能力缺口（security gap），前提是它被设计成“安全知识的放大器”和“工作流中的助手”，由开发者、安全工程师、测试、审查与确定性工具共同约束。若把 AI 当成无人看管的安全裁判，后续章节展示的幻觉、摇摆和误报会把效率收益抵消掉。

这个开场有一个重要含义：演讲的重心从抽象的“AI for security”愿景，落到工程组织怎样把安全能力移动到代码产生、审查、修复的位置。安全缺口像一条生产线上的瓶颈：代码每天被许多开发者写出，安全专家却只能抽查、咨询、建规则、处理高风险问题。AI 的价值在于把一部分安全解释、上下文整理、修复建议送到开发者手边，让瓶颈压力下降。



图 1：应用安全专家与软件开发者数量的极端不对称，是整场演讲的容量压力起点。¹

1.2 GitHub Security Lab 的可信度：它把修复放在报告之后

演讲者的团队背景很关键。Joseph 介绍 GitHub Security Lab 是一个由安全专家组成的团队，使命是保护大家依赖的开源软件；团队通过研究、教育和其他活动推进这个目标（00:01:21–00:01:33）。他随后列出 Ruby SAML 绕过研究、7-Zip heap buffer overflow 等案例，并说明团队发现并帮助修复了 1000 多个漏洞，其中 900 多个获得了独立 CVE 编号（00:01:33–00:01:53）。

Security Lab 的可信度

GitHub Security Lab 的可信度来自两层证据。第一层是漏洞研究能力：发现真实项目中的漏洞，并能把问题描述到足以获得 CVE 的程度。第二层是修复导向：演讲者强调 “we help people fix those”，这意味着团队关注从发现到修复的闭环（00:01:45–00:01:58）。在安全工程里，报告漏洞只是开始；把漏洞转化成可合并、可验证、可维护的代码改动才是真正的组织收益。

¹ 视频画面时间区间：00:02:00–00:02:43。

这里需要区分两个概念。CVE 是漏洞身份标识，像一本大型图书馆里的索引号，方便安全社区引用同一个问题。修复闭环则是工程动作，涉及补丁、回归测试、发布、依赖升级、下游通知和风险沟通。一个漏洞有 CVE，说明它被识别和记录；一个漏洞被修复，说明风险开始被实际压低。

这一背景解释了为什么演讲后面持续强调 fixing problem（修复速度问题）。如果一个团队只会制造更多安全告警，开发者会被淹没；如果一个团队能把可信发现转化成更快、更安全的修复，AI 才真正缩短了安全距离。

1.3 1:100 的压力点：安全专家无法线性覆盖开发者

演讲的核心数字出现在 00:02:03–00:02:12：如果量化安全与开发之间的缺口，约为每 100 名软件开发者对应 1 名应用安全专家。这个比例不需要被理解成精确的组织定律；它更像一个足够有力的容量模型。

可以用一个很小的公式压缩这个压力：

$$\frac{D}{S} \approx 100$$

其中：

- D ：软件开发者数量；
- S ：应用安全专家数量；
- D/S ：每名安全专家需要间接支持的开发者规模。

这个比例说明了一个硬事实：安全专家无法用人工评审的方式覆盖所有代码、依赖、配置和发布流程。把它想成医院分诊更容易理解：一名医生无法同时细看一百名病人的每个症状，所以需要问诊表、检验设备、分诊护士和标准流程先筛出风险；医生仍负责关键判断。AI 在安全工作中的理想位置也类似，它协助收集症状、解释风险、草拟修复路径，最终判断仍要回到人和验证系统。

比例的误读风险

1:100 不能被理解成“让 AI 代替 99 个专家”的许可。它说明组织必须把安全知识产品化、流程化、工具化，否则缺口会随着代码规模扩大。AI 如果缺少验证与边界，可能把错误建议扩散给更多开发者，使缺口扩大 (00:02:18–00:02:28)。

这也是本 PDF 后续章节的主线：AI 能帮助最小化 security gap (00:02:39–00:02:43)，可它需要接入可信工具、明确任务流程、人工审查和最小权限边界。单纯增加模型调用次数，不能自动创造安全能力。

1.4 Human in the loop：让 AI 提议，让人和系统负责

Joseph 在 00:02:29–00:02:39 明确说，他要展示 AI 的优点、缺点和 drawbacks，让听众能够构建 human-in-the-loop 的负责任用例。这里的 human in the loop 可以译作“人在回路中”：AI 可以观察、解释、生成候选修复，人类和确定性系统需要评估、批准、测试和追责。

人在回路中

“人在回路中”是一种控制模式。它的互补概念是 automation（自动化）：自动化提供速度和覆盖，人提供语境、责任和风险判断。一个实用的安全流程通常会把两者组合起来：AI 给出 R_{ai} (reasoning or remediation proposal, 推理或修复建议)，SAST、测试和审查提供验证，开发者对最终代码负责。

可以把 AI 输出写成一个待验证对象：

$$R_{ai} \xrightarrow{\text{review} + \text{tests} + \text{policy}} \text{Accepted fix}$$

其中：

- R_{ai} ：AI 生成的解释、风险判断或补丁建议；
- $review$ ：代码审查、安全审查或专家复核；
- $tests$ ：单元测试、集成测试、回归测试和安全测试；
- $policy$ ：组织的合规、安全基线和权限边界；
- $Accepted\ fix$ ：经过验证、可以进入代码库的修复。

这个公式只是教学压缩，来源并非演讲原文。它表达的是演讲的操作逻辑：AI 的建议不能直接等同于安全事实，必须经过审查和验证。

1.5 全局命题：缩小缺口的路径是把安全能力放进 workflow

本章最后要把开场的几条线合成一个全局命题。GitHub Security Lab 的经验提供了可信背景：真实漏洞研究、CVE、修复协作。1:100 的比例说明了组织压力：安全专家供给无法跟上开发规模。human-in-the-loop 则给出控制形态：AI 参与，人在关键节点负责。

安全价值公式

这场演讲的全局命题可以写成：

$$\text{Security value} \approx \text{Verified findings} \times \text{Workflow fit} \times \text{Human review} \times \text{Boundary control}$$

含义是：安全价值来自可信发现、合适 workflow、人工复核和边界控制的乘积。任何一项接近零，整体价值都会显著下降。

其中：

- $Verified\ findings$ ：由可靠工具、代码证据或安全研究支持的发现；
- $Workflow\ fit$ ：发现和修复建议是否进入开发者已有工作表面，例如 PR、CI、issue；
- $Human\ review$ ：人类是否审查、批准并承担工程责任；
- $Boundary\ control$ ：AI 工具是否受到权限、容器、输出校验等限制。

这一命题也暗示了一个读者容易漏掉的风险：AI 安全项目常见失败原因是没有进入正确的工程流程，而非模型缺少安全术语。安全发现如果停留在报告页里，开发者可能不会看；修复建议如果缺少测试，可能引入新 bug；智能体如果权限过宽，可能把一个小问题放大成系统性风险。

1.6 本章小结

本章建立了整份讲义的地基。Joseph Katsioloudes 以 GitHub Security Lab 的漏洞研究和修复经验开场，说明安全工作真正困难的部分是把发现变成修复。1:100 的专家与开发者比例给出容量压力，解释了为什么 AI 适合帮助缩小 security gap。可是 AI 的位置应当是 workflow 中的辅助推理层，由人、测试、确定性工具和权限边界共同约束。下一章会展示为什么这个约束必不可少：模型会 hallucinate (幻觉)，同一类问题也可能在不同上下文里产生不稳定结果。

2 写更安全的代码：幻觉、摇摆与修复问题

2.1 本节结论：原始模型输出不能单独承担安全把关

本章的核心结论是：AI 可以帮助开发者写出更安全的代码，可原始模型输出本身太不稳定，不能成为唯一安全关卡。演讲者用两个早期 demo 展示了同一个教训：模型一方面能识别真实问题，例如 SQL injection 和 input validation；另一方面会 hallucinate（幻觉），还会因为上下文变化漏掉 prototype pollution（原型污染）这类关键漏洞（00:03:41–00:05:47）。

原始输出不能直接把关

写更安全代码的实用规则是：让 AI 参与解释和修复，让更稳定的检测来源、代码审查和测试承担验证。AI 的强项是推理、总结和生成候选改动；安全门禁需要证据、重复性和责任链。

这章同时把演讲的一个中心判断提前说清楚：网络安全现在更缺的常常是 fixing speed（修复速度），而非更多 detection volume（检出数量）。Joseph 在 00:06:35–00:06:47 明确说，安全行业有许多方式发现问题，真正缺的是把这些问题更快修掉的能力。

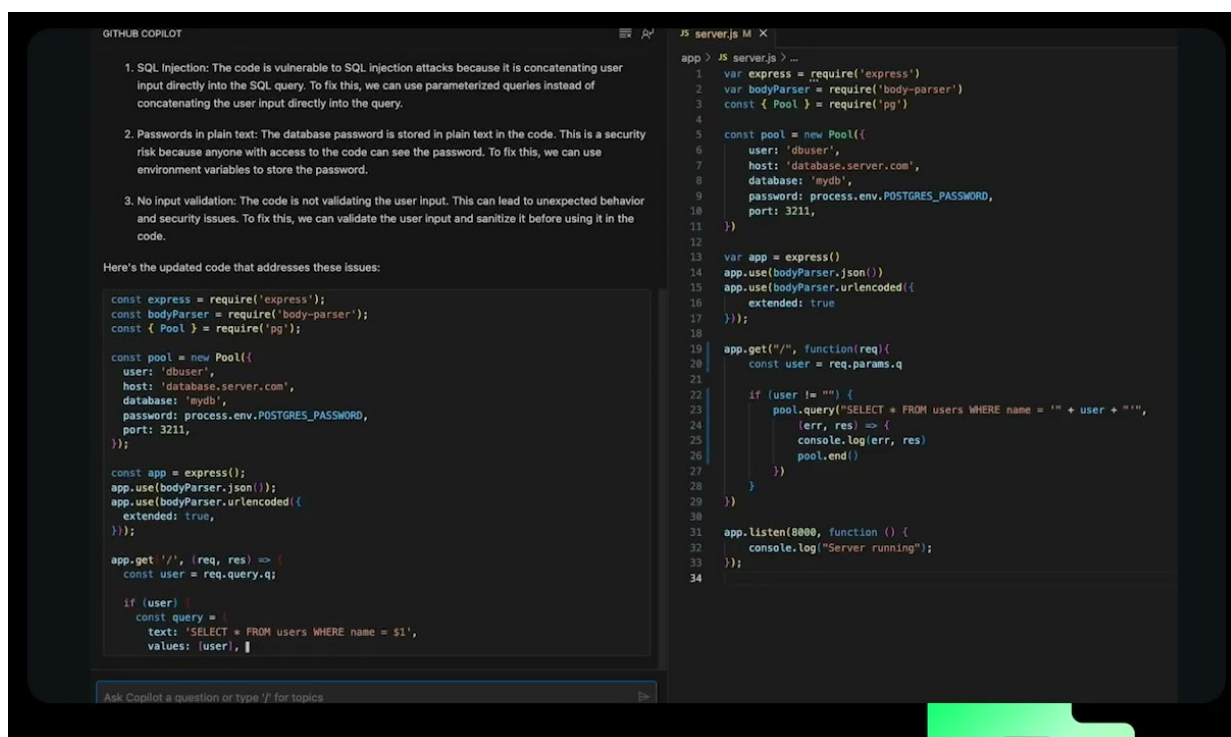


图 2: Copilot 同时识别 SQL injection 和 input validation，也生成了缺少代码证据的“Passwords in plain text”判断。²

2.2 从“shift left”到“start left”：安全必须进入代码生成现场

Joseph 先从 shift left（左移）讲起。他说自己职业生涯中一直听到资深安全领导者谈 shift left，但问题是安全左移以后，左侧仍然有缺口（00:02:47–00:03:01）。他的表达很实在：机会在于 start left，也就是从代码构建之初就接触开发者和代码生产过程（00:03:01–00:03:16）。

² 视频画面时间区间：00:03:41–00:04:40；本图为原视频帧裁剪。

Shift Left 与 Start Left

Shift left 通常指把安全活动提前到开发生命周期的左侧，例如需求、设计、编码和 PR 阶段。Start left 更进一步，强调安全能力要从代码最初形成时就出现。直观类比是质量控制：只在出厂时抽检会发现问题，在生产线一开始就给工人量具、模板和反馈，缺陷会更早暴露。

这类表述变化对应的是工作面变化。若安全反馈只在发布前出现，开发者已经切换上下文，修复成本高，争论也多。若 AI 助手能在编码、解释告警、准备 PR 修复时参与，开发者仍掌握上下文，修复路径更短。

2.3 幻觉 demo：模型说得像真的，也可能说错

第一个 demo 是一段 vulnerable code。Joseph 指出右侧代码里有一个非常简单的 SQL injection：第 22 行有用户控制变量，而且没有被 sanitize，导致第 23 行形成注入风险 (00:03:48–00:04:05)。他询问早期 Copilot：“What is wrong with this code? What are the security issues here?” (00:04:08–00:04:15)。

模型给出的三个响应中，有两个方向是有用的：SQL injection 和缺少 input validation；同时它声称存在 “passwords in plain text”，而演讲者明确说代码里没有明文密码 (00:04:19–00:04:32)。这个错误就是 hallucination：模型生成了听起来合理、带有安全术语、却没有代码证据支持的结论。

幻觉的专业外观

Hallucination 的危险在于它具有 “专业外观”。一个错误安全判断如果写得像审计报告，开发者可能花时间寻找不存在的风险，甚至修改无关代码。对安全团队来说，幻觉会消耗审查时间，稀释真正漏洞的优先级。

可以把安全判断拆成两个对象：

$$Claim = Evidence + Reasoning$$

其中：

- *Claim*：安全结论，例如 “这里存在 SQL injection”；
- *Evidence*：代码、数据流、配置、依赖版本或工具发现；
- *Reasoning*：从证据到结论的推理链。

幻觉的问题在于 *Claim* 看似完整，*Evidence* 却缺失或不匹配。安全审查不能只看模型结论，还要追问：证据在哪一行？攻击路径是什么？修复后用什么测试证明风险下降？

2.4 非确定性 demo：上下文变小，漏洞反而消失

第二个 demo 展示 non-determinism (非确定性)。Joseph 把一个含有 10 个安全漏洞的 codebase 放进上下文，再问同样的问题，模型返回 8 个结果 (00:04:41–00:05:01)。表面上可以说它有 80% accuracy，可演讲者随即指出，这个说法过于粗糙，因为结果中包含一个很关键的 prototype pollution (00:05:02–00:05:18)。

Prototype pollution 是 JavaScript 生态里很重要的风险。演讲者的解释是：当某个方法或对象从 parent method 继承内容，而 parent 被污染时，污染会向 children method 传递 (00:05:14–00:05:26)。用更直观的话说，原型像一张 “家族模板”；如果模板上被偷偷写入了恶意属性，后面按这张模板创建的对象可能继承到污染。

Prototype Pollution

Prototype pollution（原型污染）可以理解为“污染共享祖先”。在 JavaScript 中，对象会沿着 prototype chain 查找属性。若攻击者能修改共享原型，许多看似独立的对象都可能受到影响。它的对立面是局部、隔离、可控的数据写入；原型污染的难点在于影响面可能跨越许多对象。

更值得警惕的是后续变化。Joseph 因为对这个发现印象很深，就只把受 prototype pollution 影响的特定文件放进上下文，并提出同一个问题；结果这个漏洞没有出现（00:05:26–00:05:44）。上下文减少，本来应该更聚焦，可关键漏洞消失了。这就是演讲者说的“non-deterministic at its best”（00:05:44–00:05:47）。

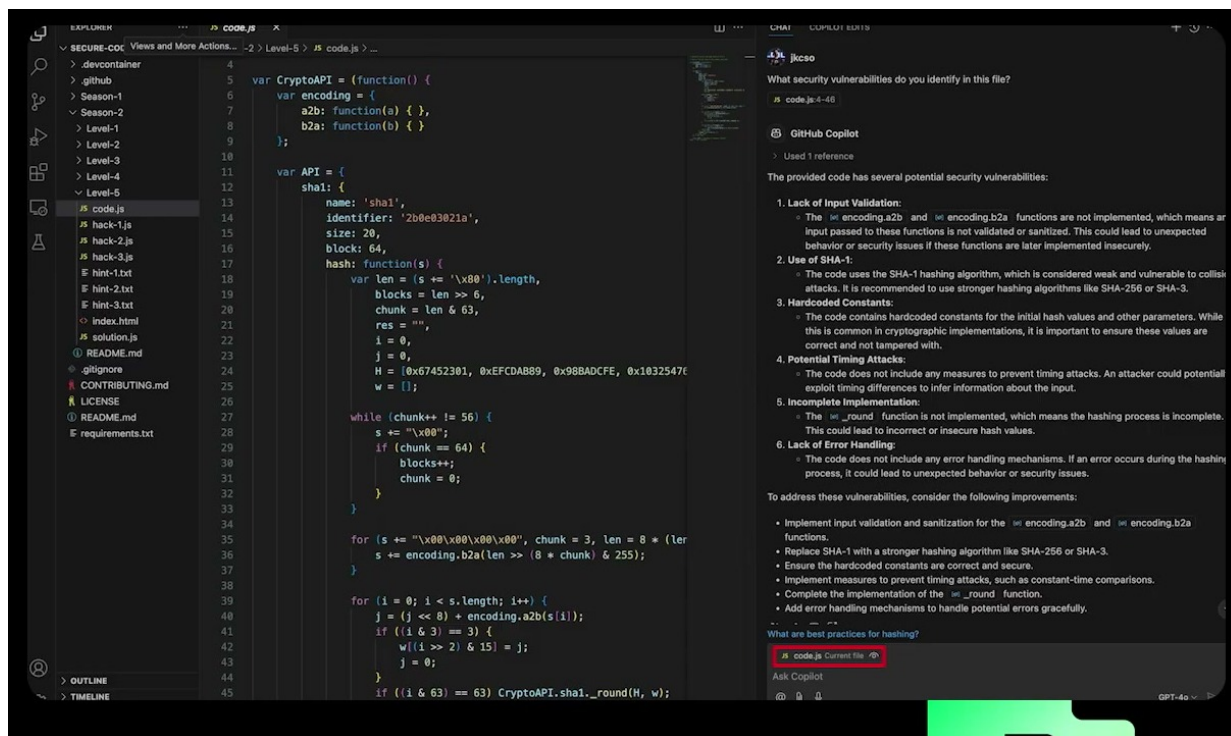


图 3：同类安全询问在更窄上下文中给出不同结果，说明模型输出需要证据和复核约束。³

非确定性的成本

非确定性意味着：同一类安全任务在不同上下文、不同模型、不同运行次数下可能给出不同结果。多跑几次也许能找出更多线索，可它会增加摩擦、成本和 false positives（误报）处理量（00:05:47–00:05:57）。

安全系统通常需要可重复性。若一个检测器今天报出关键漏洞，明天在更窄上下文里看不到它，团队就必须建立额外规则：保留证据、记录提示、对比工具发现、让审查者看到差异。AI 可以给出线索，却不能单独提供最终确定性。

2.5 换模型不能自动消除问题：需要脚手架和检测来源

Joseph 还尝试换模型，看看是否改善结果；prototype pollution 仍然没有出现（00:05:57–00:06:06）。他接着给出一个关键判断：模型当然会有帮助，可在安全任务中，更好的 scaffolding（脚手架、支撑结构）可以带来更多收益（00:06:06–00:06:19）。

³ 视频画面时间区间：00:04:41–00:05:47；本图为原视频帧裁剪。

这里的 scaffolding 可以理解为让模型工作的外部结构：确定性扫描器、代码索引、规则、任务说明、上下文选择、审查流程、测试脚本、权限边界。它像建筑工地的脚手架，工人仍然在上面工作，可脚手架决定了站位、可达范围和安全边界。没有脚手架，模型只能在大段上下文里猜；有脚手架，模型可以围绕可信发现解释原因、提出修复、生成测试。

检测先行，AI 辅助修复

本章的设计原则是：优先用更可靠的来源做 detection，再让 AI 做 reasoning layer（推理层）和 remediation helper（修复助手）。这与演讲者在 00:06:48–00:06:59 的思路一致：用其他东西做检测，再用 AI 帮助推理和修复。

可以把这个流程压缩成：

$$F_d \rightarrow R_{ai} \rightarrow Review \rightarrow Tested\ fix$$

其中：

- F_d ：deterministic finding，来自 SAST、CodeQL 或其他稳定检测工具的发现；
- R_{ai} ：AI 基于发现和代码上下文生成的解释、补丁或测试建议；
- $Review$ ：人的审查，包括代码正确性、安全影响和可维护性；
- $Tested\ fix$ ：通过测试和验证的修复。

这条链路把 AI 放在高价值位置：它减少理解成本和修复准备时间。它也避免了一个常见误区：把模型第一次回答当成完整安全结论。

2.6 修复问题：安全价值来自更快到达可验证补丁

00:06:35–00:06:47 是本章最重要的判断之一：Joseph 说网络安全的关键瓶颈集中在 fixing problem，而 detection problem 已经有大量工具和方法覆盖；行业缺少的是缩小缺口、赶上修复速度的方法。这里的 detection problem 指“发现问题”，fixing problem 指“把问题修好并安全交付”。

检测与修复

Detection 像烟雾报警器：它能告诉团队哪里可能着火。Fixing 像消防和重建：要定位火源、阻断蔓延、确认房屋还能住。报警器越多，若没有处理能力，现场只会更吵。AI 的高价值区域在于缩短从报警到修复的路径。

这个观点对工程团队很现实。安全 backlog 里堆满告警时，更多发现会制造优先级压力；如果 AI 能把一个可信 finding 转成可读解释、可审查补丁、相关测试和 PR 说明，它就直接改善了修复吞吐。后续章节关于 MCP、Skills、CodeQL、autofix 和 PR 工作流，都是围绕这个问题展开。

2.7 安全卫生：AI 不能成为唯一安全网

本章最后回到边界。Joseph 在 00:07:21–00:07:35 重申，AI 可以帮助最小化安全缺口，但 GitHub Security Lab 的观点是，它不应取代 human in the loop，也不应跳过 security testing。随后他补充，AI 正在改变 security testing 的场景，可团队仍要遵循 good security hygiene，AI 不应成为唯一 safety net（00:07:36–00:07:47）。

安全卫生

Security hygiene (安全卫生) 指那些基础却不可省略的工程习惯: 输入校验、依赖更新、最小权限、测试、审查、日志、密钥管理和发布控制。AI 可以帮助执行和解释这些习惯, 却不能替团队承担责任。把 AI 当唯一安全网, 等于把概率性系统放在确定性责任的位置上。

读者需要提前看到一个“未知未知”: AI 安全失败常常源于验证责任不清, 例如谁来验证、何时验证、用什么证据验证。若安全任务的验收标准含糊, AI 会让产出看起来更快; 真实风险可能只是被包装得更整洁。

2.8 本章小结

本章用两个 demo 说明了 AI 写安全代码的边界。第一个 demo 中, 模型识别了 SQL injection 和 input validation, 却同时编造了明文密码问题, 展示 hallucination。第二个 demo 中, 模型在完整 codebase 中找到 prototype pollution, 换成更窄上下文后漏掉它, 展示 non-determinism。演讲者由此提出更稳妥的路径: 用稳定检测来源产生发现, 让 AI 作为 reasoning layer 辅助解释和修复, 再通过 human review、testing 和 security hygiene 验证。安全的核心收益来自更快、更可靠地完成 fixing problem。

3 MCP 与 Skills: 能力加流程才可用

3.1 本节结论: 安全助手需要可信上下文, 也需要可审计流程

本节的核心结论很直接: MCP (Model Context Protocol, 模型上下文协议) 让 AI agent 接触外部工具、服务端信息和安全扫描结果, skills 则把团队希望 agent 遵守的步骤写成可审计、可维护、可扩展的流程。只有把二者叠在一起, 安全自动化才会从“会调用工具”升级为“按团队流程完成安全工作”。演讲者在‘00:07:47–00:12:40’的主线是: AI 可以作为 reasoning layer (推理层) 处理上下文问题, 但它应当连接可信来源, 并受到流程约束。

MCP 与 Skills 的乘法关系

本章的判断可以压缩成一个小公式:

$$Useful\ Security\ Agent \approx Trusted\ Context_{MCP} \times Auditable\ Process_{Skill}$$

这里的乘号表达“相互放大”的关系。只有 MCP 时, agent 可能有工具能力却缺少步骤约束; 只有 skill 时, agent 可能知道该怎么做, 却缺少足够的外部事实和执行通道。

这个观点承接上一章的警告: 模型会幻觉、会摇摆, 也可能带来可变成本。演讲者在‘00:08:49–00:09:04’说, 用 AI 直接“检测问题”会遇到非确定性和更高的美元成本; 更稳妥的定位是把它当成推理层, 围绕系统性问题、上下文问题和团队目标做分析。

3.2 MCP: 把模型从训练语料带到可信工具现场

MCP 的直觉类比是“带工牌的接口通道”。一个开发者如果只凭记忆判断生产系统状态, 很容易漏掉当前告警、仓库配置、依赖风险和扫描结果; MCP 的作用是让 agent 走到服务端工具面前, 按授权读取或调用真实系统中的上下文。演讲者在‘00:07:50–00:08:10’将 MCP 描述为帮助 AI 模型走出狭窄训练材料、访问公司和工具的 server-side information。

可信上下文

“上下文”在这里指当前系统的事实，例如仓库中已存在的 CodeQL finding、secret scanning 告警、安全公告、依赖状态、issue、PR 和组织配置。它和模型训练阶段学到的通用知识不同：训练知识像一本旧百科，MCP 提供的是工作台上的实时资料夹。

因此，MCP 的价值在于减少模型凭空猜测的空间，让推理贴近当前工具事实。设 C_t 表示 trusted context（可信上下文）， R_{ai} 表示 AI 推理结果，那么更可控的安全分析可以写成：

$$R_{ai} = f(Prompt, C_t, Process)$$

其中：

- *Prompt*：用户或工作流提出的问题，例如“这个 PR 是否引入客户端敏感数据暴露”。
- C_t ：来自可信工具的事实，例如扫描结果、服务端 finding、仓库上下文。
- *Process*：skill 规定的步骤、边界和输出格式。

这个公式只是教学压缩。它提醒读者：AI 输出的质量取决于输入事实和流程约束。若 C_t 很弱，模型容易退回猜测；若 *Process* 很弱，模型可能做出团队无法审计的动作。

3.3 MCP 的卫生规则：只把信任给已经可信的工具

演讲者在 ‘00:08:10–00:08:41’ 特别暂停，强调 MCP 的安全卫生。他假定后续讨论建立在一个前提下：只在已经信任的工具里安装 MCP server，并且避免在“野外”安装来源不明、权限范围过大的 server。

MCP 授权范围

MCP 的风险来自访问范围。若一个 agent 被授予过宽 scope，它能看到、读取、转发或改写的东西就会膨胀。最小权限原则在这里可以写成：

$$A_{granted} \subseteq A_{needed}$$

实际目标是让授权范围 $A_{granted}$ 尽量贴近任务所需范围 A_{needed} 。授权越宽，误用、泄漏和供应链攻击的后果越重。

一个简单类比：MCP server 像给外包顾问发门禁卡。只需要查阅一间会议室的资料，就发全楼层通行证，这样做会把小任务变成大风险。安全团队需要关心 server 来源、权限范围、数据流向、日志可审计性和撤权机制。

3.4 GitHub MCP：把代码安全、Secret Protection 与 Advisory 带进 agent 上下文

演讲者在 ‘00:10:03–00:10:52’ 用 GitHub MCP server 举例，因为这是他所在公司的工具。他列出的安全相关功能包括 code security、secret protection 和 security advisories。这些功能的共同作用是：从服务端拉取安全 finding，把它们送到 CLI、IDE 或其他 agent 工作环境中，让模型围绕最新上下文推理。

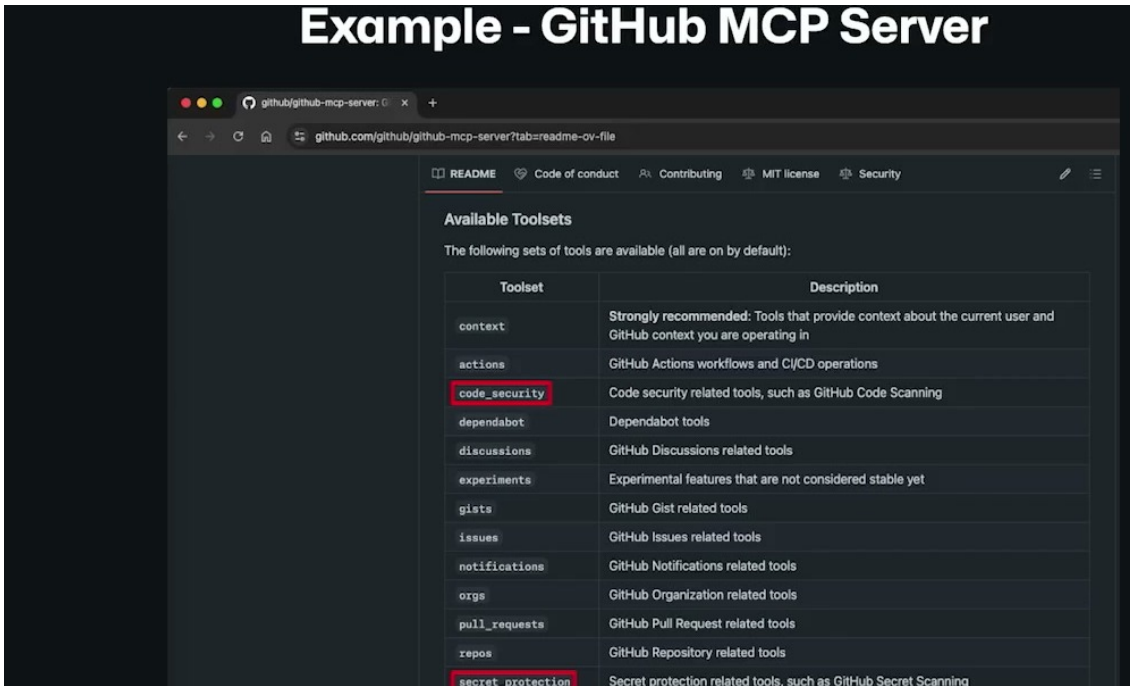


图 4: GitHub MCP server 示例：安全发现、仓库信息和平台功能通过受信任入口进入 agent 上下文。⁴

这些 finding 的关键意义在于“翻译”。演讲者在‘00:10:32–00:10:48’的表述是把服务端结果 translate into outcomes，让 AI model 或 agent 能带着更丰富的上下文推理。换句话说，MCP 在安全工作中承担一个桥接角色：

- 上游：CodeQL、secret scanning、安全公告、账户级安全状态等工具产生事实。
- 中游：MCP server 把这些事实按授权暴露给 agent。
- 下游：agent 根据团队目标，把 finding 转成解释、issue、修复建议或 PR 操作。

CodeQL 与 SAST

CodeQL 和其他 SAST（Static Application Security Testing，静态应用安全测试）工具的优势在于成熟、可重复、成本相对固定、置信度高。演讲者在‘00:09:51–00:10:02’强调这些特性仍然有价值。AI 擅长推理上下文和行为问题，SAST 擅长稳定地产生结构化 finding；两者应当组合使用。

3.5 传统静态分析覆盖模式，AI 推理覆盖行为与上下文

演讲者在‘00:09:04–00:09:24’提到一些传统工具很难回答的问题：是否使用了合适的 cryptographic primitives（密码学原语）？是否把 sensitive data（敏感数据）放到了客户端？这类问题常常需要理解业务语义、调用路径和部署位置。

同时，他在‘00:09:24–00:09:51’指出传统静态工具常以 pattern matching（模式匹配）方式工作，而很多 AI 时代的安全问题属于 behavior stuff（行为问题）。一段代码可能语法正确，也可能通过模式检查，却因为行为设计错误而泄漏内部资料。这里的重点是覆盖面的互补：

- SAST 像机场安检机，擅长稳定识别已知形状的危险物。

⁴视频画面时间区间：00:10:03–00:10:52；本图为原视频帧裁剪。

- AI reasoning layer 像经验丰富的安全审查员，能追问“这个东西放在这里是否符合业务语境”。
- MCP 像审查员手里的实时资料入口，让审查员知道仓库当前已有的 finding 和系统状态。

推理层的边界

AI 推理层带来上下文能力，也带来非确定性和可变成本。演讲者在 ‘00:08:49–00:09:04’ 明确提醒这一点。工程实践里应将 AI 输出视为候选判断 R_{ai} ，再经过 review、测试和安全验证，而非直接把它当成最终事实。

3.6 Skills：把专家流程写成可维护的 agent 操作说明

进入 ‘00:11:05–00:12:13’ 后，演讲者从 MCP 转向 skills。skills 可以理解为给 agent 的 runbook (运行手册)：遇到某类任务时，应该查什么、按什么顺序查、怎样记录证据、何时停止、怎样输出。他们的优势是 auditable、maintainable、extensible，可以放在 skill registry 中管理。

Skill 的流程价值

Skill 的安全价值在于把“专家脑中的流程”变成可检查的文本资产。团队可以审查 skill 是否要求最小权限、是否要求引用 finding、是否禁止越权操作、是否要求人类 review。相比临场口头指令，skill 更容易被版本控制、复用和改进。

这里的直觉类比是“新同事拿到工具权限”。只给新同事一串工具账号，风险很高；只给流程手册，工作推进也受限。更好的安排是：有限权限的工具入口，加上明确步骤、停机条件和交付格式。MCP 对应工具入口，skill 对应流程手册。

3.7 能力与流程的分层：MCP 负责能做什么，Skills 负责怎样做

演讲者在 ‘00:11:17–00:11:56’ 用图解释 MCP 和 skills 的关系：MCP 可以让 agent 访问 scanner results、issues 和各种功能；缺少 skills 时，agent 只有 capability (能力)，缺少团队 process (流程)；只有 skills 时，步骤可能清晰，却缺少执行所需的 power (能力)。

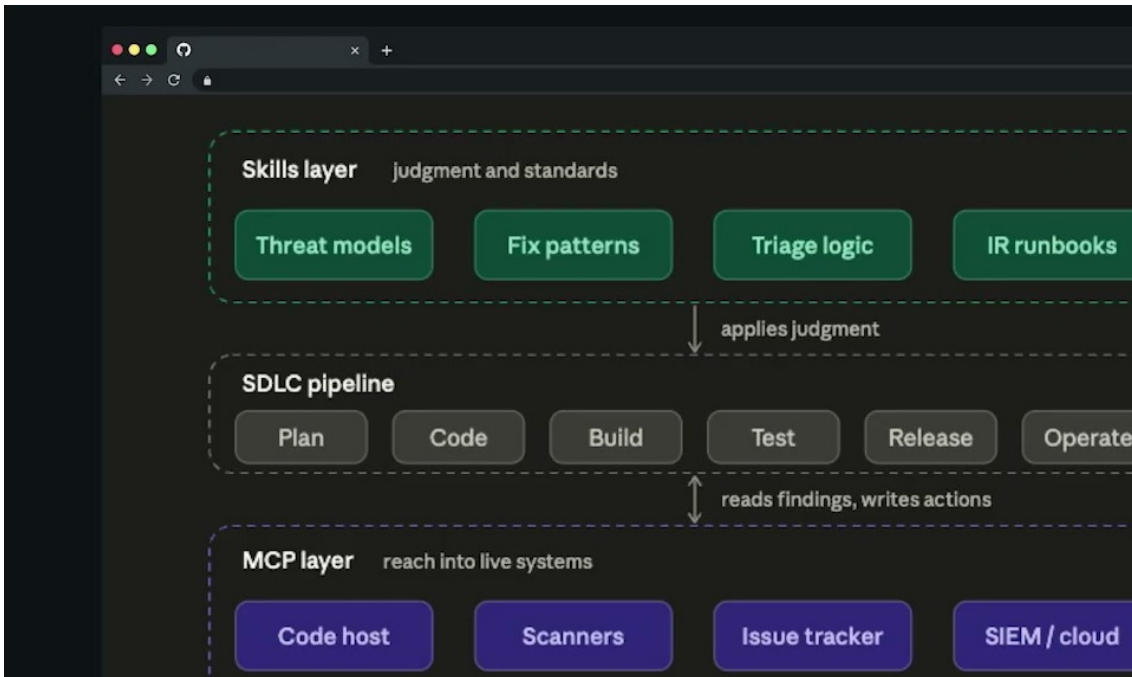


图 5: Skills layer 负责判断与标准，MCP layer 负责触达实时系统；中间的 SDLC pipeline 是二者落地的工程路径。⁵

这段可以整理成一个四层安全助手模型：

1. **事实层**：扫描器、GitHub 安全功能、secret protection、安全公告等产生 finding。
2. **连接层**：MCP server 按授权把 finding 和工具能力送进 agent 上下文。
3. **流程层**：skill 规定 triage、证据引用、修复生成、验证和交付步骤。
4. **审查层**：人类 review、CI/CD、安全测试决定结果是否可以合入。

能力与流程

“能力”和“流程”是一组互补概念。能力回答“能不能做”，流程回答“该怎样做”。安全工程里，能力扩张会放大效率，也会放大事故半径；流程约束会降低随意性，也会降低重复沟通成本。

3.8 从 finding 到 PR：本章为下一章搭桥

本章最后的时间段 ‘00:12:14–00:12:42’ 已经进入下一章主题：代码扫描发现问题，相关信息被拉取出来，issue 被创建，Copilot、Codex 或 Claude Code 这类工具可以提出修复，最后由人类在 PR 中 review，并让 CI/CD 继续把关。这里的关键转折是：MCP 和 skills 的组合价值，最终要落到开发者已有工作流里。

可审查的工程输出

安全自动化的输出应当是可审查的工程结果，而非聊天窗口里的漂亮解释。finding 需要转成 issue、patch、PR comment、测试结果或 review 任务；这些对象才会进入团队实际交付链路。

⁵ 视频画面时间区间：00:11:08–00:11:55；本图为原视频帧裁剪。

3.9 本章小结

MCP 把 agent 连接到可信工具和服务端上下文，skills 把团队流程写成可审计的操作说明。演讲者的实际主张是分层使用：让成熟扫描器提供稳定 finding，让 MCP 传递最新上下文，让 AI 负责推理和修复建议，让 skill 约束步骤，让人类 review 与 CI/CD 完成最终把关。下一章会把这套分层落实到具体场景：从 CodeQL 仪表盘到 PR 中的 autofix。

4 从仪表盘到 PR：把修复放回 workflow

4.1 本节结论：AI 修复必须进入开发者已经工作的地方

本节的核心结论是：AI 对安全修复的最大价值，来自把确定性 finding、代码上下文、解释和修复建议放回 pull request (PR) 这个开发者已经使用的工作面。演讲者在 ‘00:14:39–00:15:14’ 明确说，开发者应该在一个地方工作，这个地方应当是 PR；当修复能力进入 PR 后，GitHub 和客户在修复速度上可以达到 three times faster (三倍更快)。

PR 中的修复链路

本章的工作流可以压缩成：

$$F_d \rightarrow R_{ai} \rightarrow PR_{review} \rightarrow CI/CD \rightarrow Secure Merge$$

其中 F_d 是 deterministic finding (确定性扫描发现)， R_{ai} 是 AI 生成的解释和修复建议， PR_{review} 是人工审查和代码讨论，CI/CD 是自动化测试与安全验证。AI 位于修复链路中间，负责加速解释和改动生成；合入责任仍然落在 review 与验证环节。

这段内容继续上一章的 MCP/skills 逻辑：MCP 和工具把 finding 送到 agent 面前，skill 可以补上 triage logic (分诊逻辑)，PR 则成为开发者真正采取行动的地方。

4.2 旧 workflow：CodeQL 仪表盘告诉你哪里错了，却离修复还有距离

演讲者在 ‘00:13:02–00:13:39’ 回顾 “AI 之前” 的常见路径：开发者或安全人员进入 dashboard，例如 GitHub 的 CodeQL 仪表盘，看到代码问题列表、严重程度和解释。这种方式有价值，因为它把问题检测出来，也给出标准化说明。

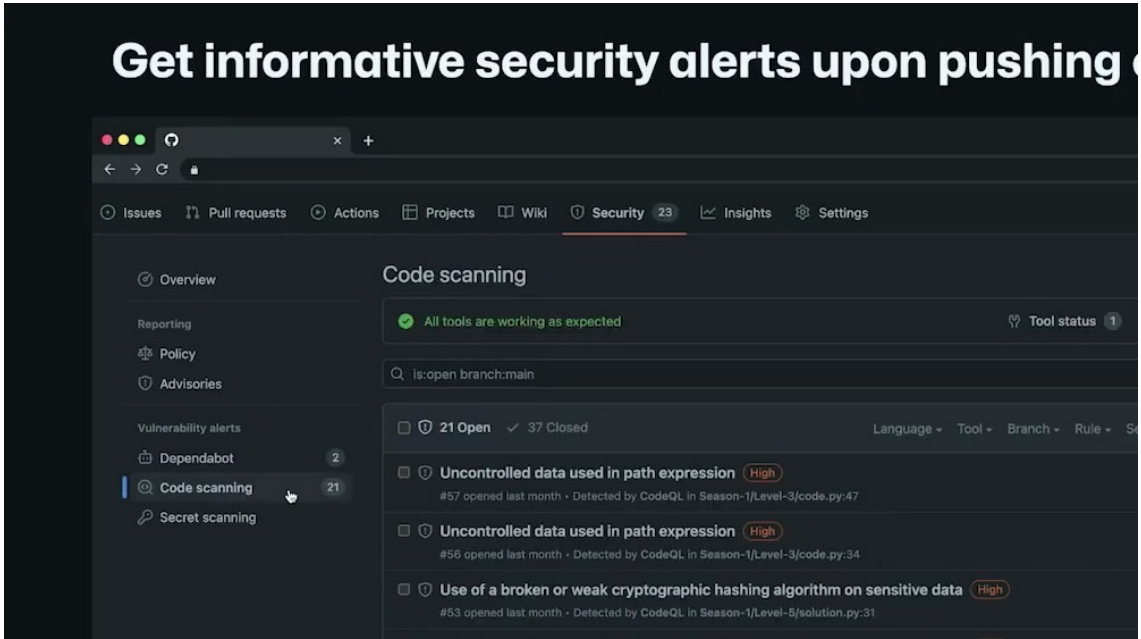


图 6: 传统安全告警界面能稳定列出 findings，但解释和修复路径仍可能离开发者的 PR 工作面较远。⁶

问题在于，这类解释通常比较通用。演讲者在 ‘00:13:25–00:13:32’ 说，解释 always the same，并且不够 specific to your code。也就是说，仪表盘回答了“这里有风险”，却没有完整回答“在这个 PR、这个函数、这个业务约束下，最小改动是什么”。

CodeQL 与体检报告

CodeQL 属于 SAST (Static Application Security Testing, 静态应用安全测试) 体系。它的优势是 repeatable (可重复)、mature (成熟) 和 high confidence (高置信度)。它像一张稳定的体检报告，能指出异常指标；开发者还需要医生式解释和治疗方案，才能把异常转成具体修复。

把 detection (检出) 和 fixing (修复) 拆开看，旧仪表盘 workflow 主要优化 F_d ，即“找到了什么”。AI autofix workflow 进一步优化 T_{fix} ，即从 finding 到安全补丁所需的时间：

$$T_{fix} = T_{understand} + T_{patch} + T_{review} + T_{validate}$$

其中：

- $T_{understand}$: 理解 finding 的时间。
- T_{patch} : 写出候选补丁的时间。
- T_{review} : 人工审查和讨论时间。
- $T_{validate}$: 测试、CI 和安全验证时间。

AI 的主要作用是压缩前两项，同时为后两项提供更清楚的上下文。

4.3 新 workflow: Autofix 把解释和候选代码带到 PR

从 ‘00:13:40–00:14:28’ 开始，演讲者展示当前方式：fixes are coming to the PR，这里指 Copilot autofix，也可以由其他 AI 系统实现。AI 解释基于 deterministic findings from the SAST tools (来自 SAST 工具的确定性 finding)，并在 PR 级别给出高亮原因、候选代码和直接提交或编辑的入口。

⁶ 视频画面时间区间：00:12:45–00:13:44；本图为原视频帧裁剪。

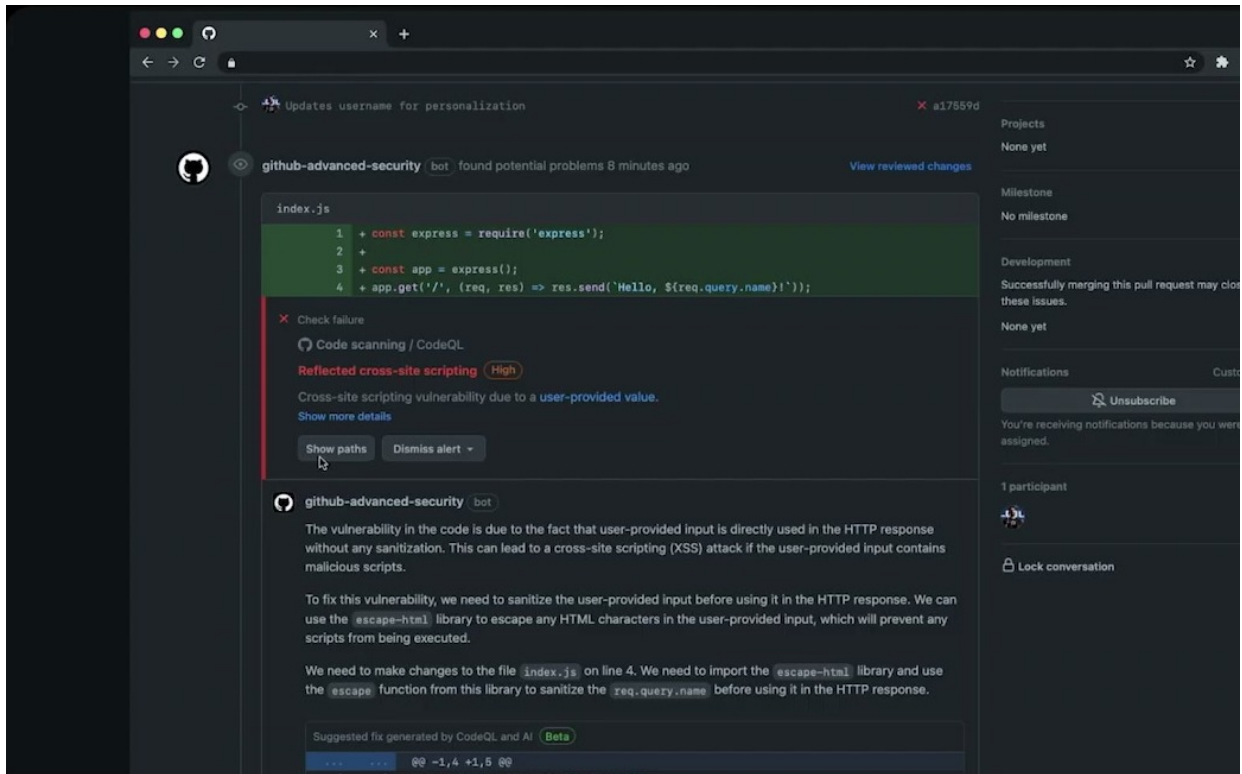


图 7: PR 中的 CodeQL finding、风险解释和候选修复放在同一个工作面，降低了理解与切换成本。⁷

这比单独的仪表盘更接近开发者心智。开发者打开 PR 时，已经在看 diff、上下文、测试状态和 review comment；安全修复建议出现在同一个表面上，开发者无需额外切换到另一个网站再把信息手工搬回代码。

PR 级 Autofix

PR 级 autofix 的关键并非“AI 自动合入代码”。关键是把扫描证据、代码解释、候选补丁和审查入口放在同一个工作面上，让开发者更快理解、更快修改、更容易验证。

这里也能看到 MCP 与 skills 的分工：MCP 或平台能力负责带入 finding 和代码上下文；skill 可以规定分诊逻辑，例如哪些 severity 先处理、哪些文件需要额外测试、什么时候只生成建议、什么时候允许提交候选 patch。

4.4 PR 是工作面：减少工具切换就是减少安全摩擦

演讲者在‘00:14:39–00:15:06’给出一个来自四年半 GitHub 工作经验的判断：开发者需要在一个地方工作。可以有 Codex、GitHub Copilot app 等独立应用，但当开发者需要登录 X 网站、Y 网站，再把信息搬回自己的交付链路时，摩擦会增加，长期不可持续。

工作面

“工作面”（work surface）指一个人完成任务时主要停留的界面和对象。对开发者而言，PR 同时包含 diff、discussion、review、CI、branch、commit 和合入按钮。安全建议进入 PR，就像维修建议直接贴在机器旁边；若建议留在另一个系统里，执行者需要来回搬运信息。

摩擦可以粗略表示为额外切换成本：

$$F_{workflow} = N_{tools} \times C_{switch} + C_{copy} + C_{context\ loss}$$

⁷ 视频画面时间区间：00:13:44–00:14:28；本图为原视频帧裁剪。

其中：

- N_{tools} ：完成修复需要切换的工具数量。
- C_{switch} ：每次登录、跳转、找上下文的成本。
- C_{copy} ：把 finding、解释和代码片段搬运回 PR 的成本。
- $C_{context\ loss}$ ：切换过程中丢失代码语境和审查语境的成本。

AI 安全产品若希望真正提升修复率，就必须降低这个摩擦项，而 PR 正是最自然的降摩擦位置。

4.5 三倍更快：速度来自链路缩短，仍需验证

演讲者在 ‘00:15:06–00:15:14’ 提到，将修复放在 PR 中后，GitHub 和客户在 fixes 上达到 three times faster。紧接着在 ‘00:15:14–00:15:23’ 他再次强调：安全领域有 fixing problem，而检测已经相对充足；GitHub 在两周内修复了 600 个漏洞，并把 fixed rate 拉高，客户也取得类似效果。

速度主张的边界

“三倍更快” 应理解为特定工作流和客户实践中的经验性主张，不能机械外推到所有组织。修复速度还取决于代码库规模、测试覆盖、review 文化、漏洞类型、权限边界和 CI/CD 成熟度。AI 建议进入 PR 会缩短路径，却不会自动消除工程验证成本。

更准确的读法是：AI autofix 改善了从 F_d 到候选补丁的路径，尤其压缩 $T_{understand}$ 和 T_{patch} 。若组织没有 review 纪律、测试能力和安全验证，速度提升可能转化为更快地产生未经验证的改动。

4.6 人工 Review、CI/CD 与安全验证仍是合入门槛

本节开头的跨章片段 ‘00:12:39–00:12:46’ 很重要：演讲者说人类进入 review，以便 release 并保持 CI/CD。也就是说，PR autofix 的闭环应理解为 “AI 提议后进入正常工程门禁”。

合入前检查

PR 中的 AI 修复建议应当满足三类检查：

- **代码正确性**：补丁是否保持功能语义，是否破坏边界条件。
- **安全有效性**：补丁是否真正移除 finding，是否引入替代风险。
- **工程可合人性**：测试、CI/CD、review comment 和团队规范是否通过。

这也是 “human in the loop” 的实际形态。人类带着业务语境、风险偏好和责任判断审查候选变更。CI/CD 则提供机器可重复验证：编译、测试、静态检查、策略检查和部署前门禁。

4.7 从本章看全局：AI 安全价值来自修复链路设计

将本节放回整场演讲，逻辑很清楚：安全团队已经拥有很多 detector，问题在于 finding 没有足够快、足够稳地转成 reviewed fix。MCP 把可信 finding 带进 agent 上下文，skills 规定处理步骤，autofix 生成候选改动，PR 承载审查和合入，CI/CD 负责验证。

责任链条

一个常见误解是把 PR autofix 当作“替代安全工程师”。更准确的理解是，它把安全工程师的部分解释劳动和补丁草拟劳动前移到开发者工作面。责任链条仍然存在：工具提供证据，AI 提出候选，开发者和 reviewer 判断，CI/CD 验证。

4.8 本章小结

CodeQL/SAST 仪表盘能稳定检出问题，但通用解释和独立页面会拉长修复路径。AI autofix 的改进点在于把确定性 finding、代码特定解释和候选补丁放到 PR 中，让开发者在熟悉的工作面完成理解、修改、review 和验证。演讲者的“三倍更快”主张应按 workflow 改善来理解：速度来自减少切换、缩短理解与补丁生成时间，并且仍以人工 review、CI/CD 和安全验证作为合入门槛。

5 Agentic Workflows 与 Task Flows：把专家知识写进自动化

5.1 本节结论：智能体要像安全作业一样运行

本节的核心结论是：**agentic workflow** 的价值在于把安全专家的判断条件、触发时机和操作边界写进仓库自动化。它适合承担那些需要结合项目语境、公司规则和代码级判断的安全任务；同时，它仍然需要 SAST、CodeQL、测试、人工复核等稳定机制提供证据和边界。演讲者在 ‘00:15:27–00:15:46’ 先抛出一个现实问题：既然安全团队开始使用 agentic workflows，为什么还需要 SAST tooling 或其他 security tools？他的回答延续前文对 non-determinism 与成熟度问题的提醒：AI 的优势是 tailoring，稳定的安全工具仍然承担可重复、可审计的底座。

Agentic workflow 的一句话定义

在本章语境中，**agentic workflow** 是一种运行在仓库里的安全自动化作业：它带着项目规则、触发条件和权限边界，在特定时机检查代码、生成 issue 或提出修复建议。可以把它理解为“会推理的 CI job”：CI job 提供确定的执行环境，agent 提供对上下文的解释和取舍能力。

这个定义有一个关键含义：智能体的质量取决于三类输入的组合。第一类是确定性发现，例如 SAST 或 CodeQL 给出的 finding；第二类是项目上下文，例如哪些目录越界、哪些风险对团队重要；第三类是专家流程，例如安全研究会按什么顺序审查一个仓库。若用一个压缩公式表示，本节的自动化收益可以写成：

$$W_{sec} \approx T \times C \times R$$

其中：

- W_{sec} ：安全工作流带来的有效安全产出；
- T ：触发条件的准确性，例如 schedule、pull request、issue、依赖变更或代码路径变化；
- C ：上下文质量，包括仓库规则、公司约束、历史 finding、测试结果和相关文件；
- R ：可复用的专家规则，也就是 task flow 或 skill 类指令。

这个公式属于对 ‘00:15:27–00:19:50’ 这段内容的教学压缩，演讲中没有直接给出。它提醒读者：只提高模型能力，不能自动提高安全结果；触发、上下文和规则任何一项薄弱，都会拖低最终产出。

5.2 为什么 SAST 仍然要留在体系里

演讲者在 ‘00:15:33–00:15:46’ 预判了一个常见误解：如果 agentic workflows 可以做安全检查，传统 SAST tooling 是否就失去意义？他的回答很直接：前面已经展示过 AI 的 non-determinism、成熟度限制和幻觉风险，所以安全工具仍然必要。SAST 的作用像体检中的基础化验：它可能覆盖不了全部风险，却能稳定地把已知模式、语义规则和历史经验转成可重复的 finding。

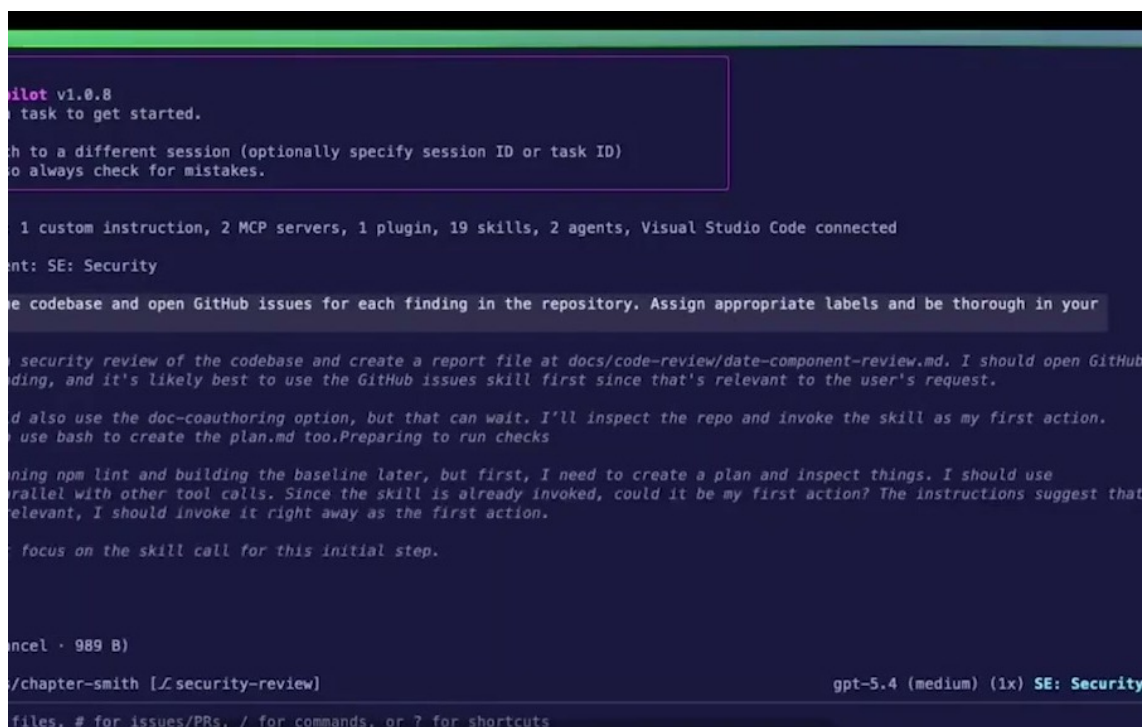
SAST 与 agent 的互补关系

SAST (Static Application Security Testing) 更擅长稳定识别已建模的问题，例如数据流、注入路径、危险 API 使用或已知规则违反。Agent 更擅长把 finding 放进项目语境里解释：这个风险是否重要、应由谁修、修复是否影响业务逻辑、是否需要开 issue 或直接发起代码修改。两者像“雷达”和“导航员”：雷达给出稳定信号，导航员结合目的地和路况做行动建议。

因此，本节不能把 agentic workflow 写成替代式叙事。更准确的结构是：SAST 负责提供可复查的底层证据，agentic workflow 负责把这些证据同项目上下文、触发条件和修复动作连接起来。这个连接回应了前文反复出现的主线：行业更缺的是 fixing throughput，而仅扩大 detection volume 容易制造更多排队等待处理的警报。

5.3 仓库里的安全智能体：从脚本规则到触发式运行

在 ‘00:15:53–00:16:27’，演讲者展示了一个 basic example：一个 security agent 被设置起来，指令中写明 out of scope、团队关心什么、公司如何处理某些问题。随后 agent 在代码层面识别 “only by AI” 才能挑出的漏洞，并让团队选择创建 issue 或在代码开发处直接修复。这里的重点是 “tailoring”：智能体被调成适合这个组织、这个仓库、这类风险的自动化安全参与者。



```
pilot v1.0.8
> task to get started.

...
to a different session (optionally specify session ID or task ID)
to always check for mistakes.

...
1 custom instruction, 2 MCP servers, 1 plugin, 19 skills, 2 agents, Visual Studio Code connected

Agent: SE: Security

...
the codebase and open GitHub issues for each finding in the repository. Assign appropriate labels and be thorough in your

...
security review of the codebase and create a report file at docs/code-review/date-component-review.md. I should open GitHub
finding, and it's likely best to use the GitHub issues skill first since that's relevant to the user's request.

...
I'd also use the doc-coauthoring option, but that can wait. I'll inspect the repo and invoke the skill as my first action.
I use bash to create the plan.md too. Preparing to run checks

...
ning npm lint and building the baseline later, but first, I need to create a plan and inspect things. I should use
parallel with other tool calls. Since the skill is already invoked, could it be my first action? The instructions suggest that
relevant, I should invoke it right away as the first action.

...
focus on the skill call for this initial step.

...
ancel · 989 B)

...
/chapter-smith [Lsecurity-review] gpt-5.4 (medium) (1x) SE: Security

...
files, # for issues/PRs, / for commands, or ? for shortcuts
```

图 8: Agentic workflow 把仓库任务、安全规则和触发条件写入可运行脚本，使 agent 像安全作业一样执行。⁸

这张图把 agentic workflow 的运行方式压缩成三层：输入层来自仓库和触发事件，规则层由 agent

⁸ 视频画面时间区间：00:15:27–00:17:16；本图为原视频帧裁剪。

指令与项目约束组成，输出层落到 issue、修复建议或 PR 内动作。读者要关注的重点是“流程边界”：agent 的价值来自它能读取代码和上下文，也来自它被脚本、权限和审查动作限制在可追踪范围内。

在 ‘00:17:29–00:18:00’，演讲者进一步把 workflow 放回仓库环境：团队可以直接在 repo 里创建 agent workflow；这些 workflow 可以按 schedule 运行，也可以理解什么事件会触发自己；运行环境与 GitHub Actions 使用相同类型的 VM。这个描述把 agentic workflow 从“会聊天的模型”拉回到工程自动化：它像一个有推理能力的仓库作业，等待触发，读取上下文，执行既定程序。

触发条件是安全自动化的第一道设计题

安全 agent 什么时候运行，和它运行时做什么同样重要。定时运行适合周期性巡检；事件触发适合 pull request、依赖变更、敏感文件改动、issue 标签变化等场景。触发过宽会增加成本和噪声，触发过窄会漏掉关键时机。

从工程角度看，触发式运行还带来审计收益：每次 agent 执行都有明确入口，例如某次 PR、某个 schedule、某个 issue 状态变化。安全团队可以追溯“为何运行、读取了什么、产出什么、谁批准了修复”。这比临时聊天式询问更适合生产安全流程。

5.4 指令设计的成本：上下文膨胀会让流程变钝

演讲者在 ‘00:16:49–00:17:22’ 给出一个很实用的经验：如果把团队所有 instruction、rule 和约束都塞进同一个 script，AI context 会被 bloating，流程效果会变差。他的处理方式是分层：有些内容进入 agents，有些进入 instruction files，有些交给 tooling。这是本节最值得展开的工程判断。

Context bloat 的真实风险

上下文窗口像工作台。把所有螺丝刀、图纸、合同、日志、历史邮件都堆在同一张桌上，表面上信息更多，实际会让关键线索更难被抓住。对 agent 来说，过长、过杂、边界不清的指令会增加遗漏、误解和自相矛盾的概率。

一个更稳的分层方式是：

- **Agent 指令层**：写角色、目标、边界、输出格式和必须遵守的安全原则；
- **文件规则层**：写项目局部约束，例如某目录的 threat model、某服务的数据分类、某语言栈的常见漏洞；
- **工具层**：承载可重复的检查，例如 CodeQL、测试、依赖扫描、secret scanning 和格式校验；
- **人工审核层**：处理业务风险、误报取舍、权限批准和高影响修复。

这个分层的直观标准是：稳定、可计算、可重复的规则优先交给工具；需要项目语义和安全经验的判断交给 agent；需要责任承担和风险取舍的动作交给人工。这样可以减少上下文噪声，也能让每个层次的失败更容易定位。

5.5 Task Flows：把安全研究员的审查路线公开出来

在 ‘00:18:34–00:19:50’，演讲者转向更强的模型和 GitHub Security Lab 的研究经验。他明确说，强模型不会“自己”发现漏洞，仍然需要 security people steering them，并把安全研究员的知识 codify 成 task flows。这里的 **task flow** 可以理解为专家审查路线：先看什么、如何缩小范围、哪些迹象提示风险、发现异常后怎样继续验证。

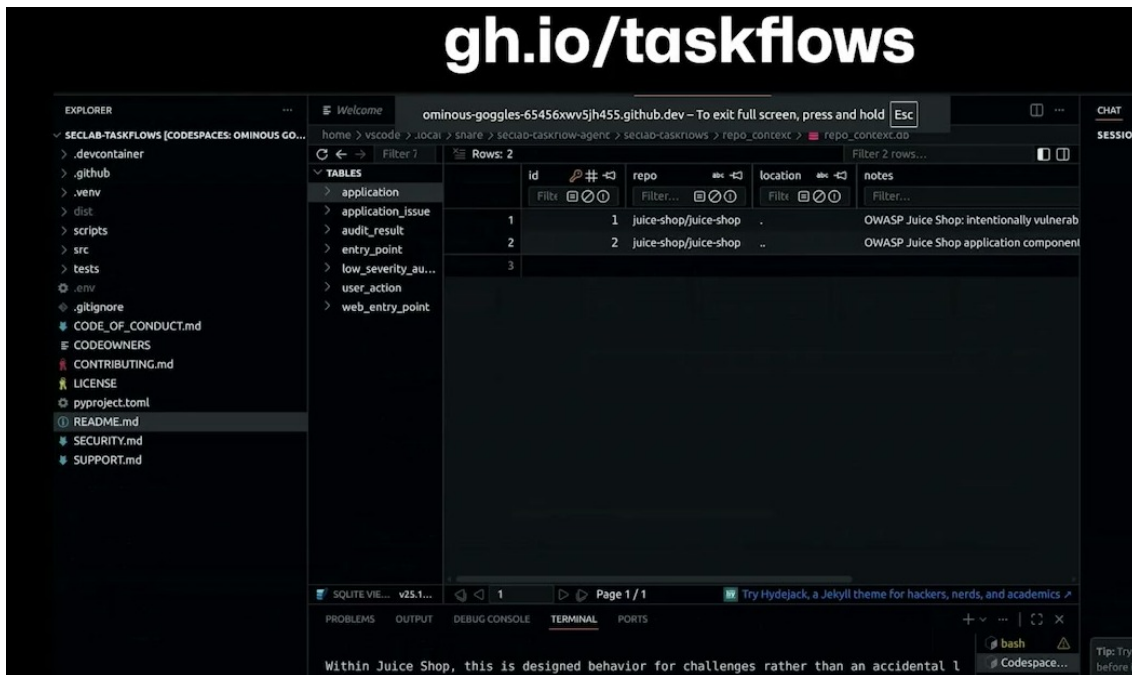


图 9: gh.io/taskflows 展示了把安全研究员的审查路线写成可复用 task flows 的开放入口。⁹

这张图保留了演讲中出现的开放入口 gh.io/taskflows。它的作用是把 task flow 从抽象概念落到可访问资源：读者可以把它理解为一套公开的安全审查路线库，模型沿着这些路线读仓库、定位风险、形成可复查的操作步骤。

Task flow 与普通 checklist 的区别

普通 checklist 往往只告诉读者“检查 X、检查 Y、检查 Z”。Task flow 更接近专家的路线图：它包含顺序、分支、上下文读取方式、验证动作和退出条件。类比代码审查，新手只会逐行看 diff；专家会先定位信任边界、数据入口、权限转换和异常处理，再决定哪些代码需要深挖。

演讲者给出的源指针是 gh.io/taskflows，并说明它免费、开源，目标是把安全研究员发现漏洞的知识开放出来。这个点很重要：模型能力只是执行器，安全研究知识才是方向盘。若没有 task flow，强模型可能在大仓库里漫游；有了 task flow，它能沿着研究员总结的路径执行 manual security review，并在 Codespaces 之类环境中自动化运行。

本节的操作性结论

Agentic workflow 负责“何时运行、在哪运行、产出什么”；task flow 负责“按什么专家路径看问题”。二者结合后，安全团队可以把稀缺的人工经验变成可复用、可审计、可持续改进的仓库自动化。

5.6 风险边界：自动化越聪明，越要留下人和工具的刹车

本节材料容易让读者产生过高期待，所以需要把边界说清楚。演讲者在‘00:16:39–00:16:48’使用“deterministically as much as possible”这样的表达，说明 agent 会尽量按脚本执行，却仍然受模型不确定性影响。这里的“as much as possible”就是风险提示：安全工作流可以变得更稳，不能承诺完全确定。

工程上应保留三类刹车。第一，使用 SAST、测试和依赖扫描给出可重复证据。第二，把 agent 输出落到 issue、PR、review comment 等可审计对象中。第三，对高影响修复保留人工审批和 CI 验证。

⁹ 视频画面时间区间：00:18:35–00:19:34；本图为原视频帧裁剪。

这样，agentic workflow 才能成为安全吞吐量的放大器，而不会变成难以追责的自动化风险源。

5.7 本章小结

本章说明了 agentic workflows 与 task flows 的分工：前者把安全 agent 放进仓库运行机制，后者把安全研究员的审查知识写成可复用流程。SAST 和其他安全工具仍然提供稳定证据；agent 则把证据、上下文和行动连接起来。读者应记住三点：触发条件决定 agent 何时介入，指令分层决定 agent 是否清醒，task flow 决定 agent 是否沿着专家路线工作。源材料中的 [gh.io/taskflows](https://github.com/gh.io/taskflows) 是本章最具体的资源指针，代表“把专家审查路线开放出来”的实践入口。

6 供应链决策：更快地研究依赖风险

6.1 本节结论：AI 的作用是缩短研究时间，风险决定仍由人承担

本节的核心结论是：AI 可以把依赖评估从漫长的信息搜集压缩成带证据链接的 executive summary，但是否采用某个依赖，仍取决于团队的 risk appetite、业务场景和人工判断。演讲者在 ‘00:19:51–00:20:01’ 把主题切到 supply chain decisions：开发者或 agent 如何做出更 informed 的供应链决策。随后他用一个很现实的问题打开场景：团队在决定是否引入一个依赖时，通常会花多少时间，三分钟、三十分种，还是更久？这个提问对应的是日常工程里最常见的安全盲点：依赖被快速加入，风险研究滞后发生。

供应链决策的主问题

依赖评估并不是“这个库有没有漏洞”这么单一。更准确的问题是：在当前项目、当前数据敏感度、当前维护能力和当前上线压力下，引入这个依赖是否划算。AI 可以帮助整理信号，人需要决定可接受风险。

可以用一个简化公式表达依赖采用前的风险判断：

$$R_{dep} = I \times P \times E$$

其中：

- R_{dep} ：依赖风险的粗略量级；
- I ：风险影响，包含可被攻击的数据、权限、用户规模和业务后果；
- P ：问题发生概率，受项目维护质量、已知漏洞、代码复杂度和暴露面影响；
- E ：证据不确定性，来源越少、越旧、越难验证， E 越高。

这个公式同样是教学压缩，不是演讲原文。它的价值在于提醒读者：一个依赖在内部脚本中可能可接受，在处理支付、身份或生产权限的服务里可能超出风险承受范围。

6.2 从三分钟问题开始：依赖引入前到底该研究什么

在 ‘00:20:20–00:20:36’，演讲者连续追问听众：决定是否把某个东西引入代码时，花超过三分钟的人有多少？超过三十分钟的人有多少？这个问题听起来像现场互动，实际指出了供应链安全的日常形态。多数依赖决策往往发生在开发者为了完成一个功能、修一个问题、试一个框架的过程中，正式安全评审会经常来得更晚。

快速引入依赖的隐藏成本

依赖进入项目后，风险会沿着更新、构建、运行时权限和传递依赖继续扩散。最初节省的十分钟，可能在漏洞响应、版本冲突、许可证审查或 incident 处理中变成数小时甚至数天。AI 摘要能节省研究时间，却不能替团队承担长期维护责任。

因此，依赖评估至少要覆盖五个问题：

- **维护状态**：项目是否活跃，是否有稳定发布节奏，核心维护者是否仍在响应问题；
- **已知漏洞**：是否有公开 CVE、security advisory、未修复 issue 或高风险历史；
- **行为暴露面**：依赖是否处理输入、网络、文件系统、模板、认证、加密或执行外部命令；
- **传递风险**：它会带入哪些 transitive dependencies，这些依赖是否扩大攻击面；
- **证据链接**：每个结论是否能回到具体 URL、issue、advisory、release note 或源码位置。

这五项构成了本节后续的 repeatable risk checklist。它们也说明了 AI 摘要最该做的事：把信息组织成决策材料，而非只给一个“可以用”或“不要用”的断言。

6.3 gh.io/risk：开放 instruction files 与 executive summary

在 ‘00:20:36–00:21:10’，演讲者介绍团队为加速研究准备了四个 instruction files，用户可以扩展；这些文件免费且开源，URL 是 gh.io/risk。它们的目标是生成一个 executive summary，展示用户需要小心的地方，并使用团队认可的信息源，同时让用户能点击链接自行验证。

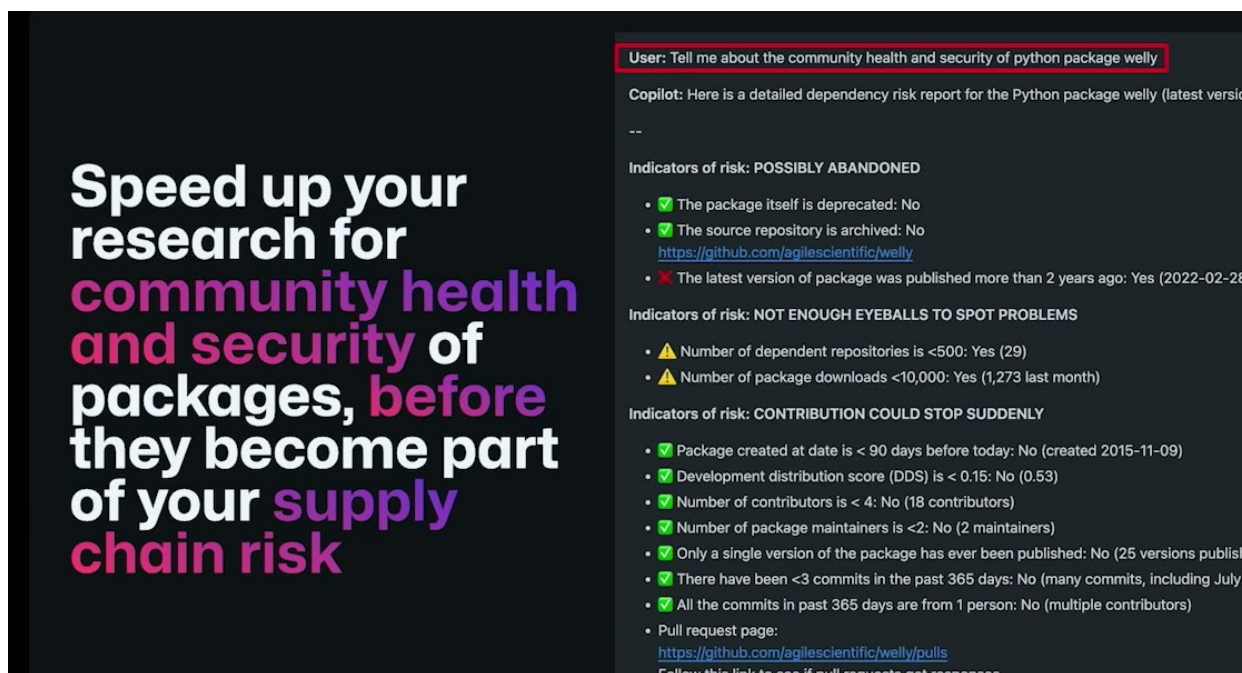


图 10: gh.io/risk 示例把依赖研究整理为带证据入口的摘要，帮助团队更快进入供应链风险判断。¹⁰

这张图展示的是演讲中的供应链风险评估示例，重点在于示范 gh.io/risk 如何把风险项、证据入口和 executive summary 放在同一个决策页面。读者不应把它理解成 Bootstrap 的最终安全结论；它更像一个“研究入口”：摘要先帮助人定位问题，随后仍要顺着链接、源码和项目场景继续验证。

¹⁰ 视频画面时间区间：00:20:37–00:21:10；本图为原视频帧裁剪。

Executive summary 在安全决策中的作用

Executive summary 的作用，是把决策者最先需要的信息放在前面：结论、主要风险、证据链接、需要人工确认的点，并保留复杂问题的证据入口。对依赖评估来说，它像体检报告首页：先告诉读者哪些指标异常，再引导读者查看具体化验单。

gh.io/risk 的意义在于把“如何问、看哪些源、如何组织摘要”写成开放 instruction files。它继承了上一章 task flows 的思路：专家知识需要被编码，模型才更容易沿着正确路线研究风险。这里的“开放”也很关键。开放 instruction files 允许团队审查指令、扩展规则、调整输出风格，并根据自己的 risk appetite 修改评估重点。

6.4 Bootstrap 示例：问清楚“别人会怎样攻击我”

在‘00:21:25–00:22:15’，演讲者展示了同一类评估可以在 CLI 中运行，也可以在 web 上可视化。他选用 Bootstrap 这个开源项目作为例子，因为它是一个正在考虑纳入 supply chain 的项目。他向系统提出的问题大意是：如果我使用这个项目，需要小心什么？演讲者补充说，这也是他们在 open source project office hours 中经常被问到的头号问题：How can somebody hack me?

这个问题比“这个库安全吗”更有操作性。因为安全决策需要具体攻击路径：攻击者能通过模板、样式、构建脚本、依赖更新、文档示例、插件机制或供应链投毒影响项目吗？如果只问一个抽象安全评分，模型可能给出漂亮却难以行动的答案；如果问“别人怎样攻击我”，系统更容易沿着入口、资产、权限和影响去组织材料。

好的依赖风险问题

把问题从“这个项目安全吗？”改成“在我的项目里使用它时，攻击者可能通过哪些路径影响我？”前者容易得到泛化评价，后者更容易产出可审查的 threat model、证据链接和缓解动作。

演讲者在‘00:22:01–00:22:15’还强调：不同模型跑出的结果相似，它像一个安全人员告诉你开始使用这个项目时要小心什么，使你无需亲自审完所有代码，并能跳到 specific URLs 节省时间。这里应谨慎表述为“相似”和“节省研究入口成本”，不能夸大成模型已经替代完整审计。

6.5 可复用依赖风险清单：从摘要回到证据

基于演讲中 gh.io/risk 和 Bootstrap 的示例，可以把依赖采用前的研究流程整理成一张可复用清单：

1. **定义采用场景**：说明依赖会被用于前端、后端、构建、测试、CLI、生产服务或内部工具；
2. **识别暴露面**：标出它是否处理用户输入、网络请求、认证、模板渲染、文件读写、插件加载或代码执行；
3. **检查维护信号**：查看 release 节奏、维护者响应、issue 堆积、最近 security fix 和版本兼容策略；
4. **检查已知安全信号**：查看 CVE、GitHub Security Advisories、release notes、公开 exploit、未合并安全 PR；
5. **检查传递依赖**：列出新增依赖树，关注深层依赖中的高权限包、废弃包和维护停滞包；
6. **输出 executive summary**：给出主要风险、证据链接、建议缓解措施和需要人工确认的问题；
7. **按 risk appetite 决策**：把结论放入团队的风险承受范围里，决定采用、延后、替换、加隔离或追加审查。

摘要质量受信息源质量限制

AI 生成的供应链风险摘要依赖输入源。如果信息源过旧、链接失效、项目迁移、漏洞尚未公开，摘要可能显得完整却遗漏关键风险。高敏感系统应把 AI 摘要当成研究入口，再结合安全 advisory、源码审查、SBOM、依赖扫描和人工判断。

这张清单也适合交给 agentic workflow 执行：当 PR 新增依赖时自动触发；agent 读取包名、版本、用途和相关文件；工具收集 advisories、release notes 和 dependency tree；模型生成 executive summary；开发者或安全人员根据 risk appetite 做最终决定。

6.6 Risk appetite: 同一个依赖，在不同系统里的答案可能不同

演讲者在 ‘00:20:02–00:20:18’ 提到自己的 risk appetite 与当前 approach 对齐，质量达到可以在现场展示并有信心 shipping 的程度。这个细节值得保留，因为它防止读者误把依赖风险评估理解成统一评分。Risk appetite（风险胃口、风险承受度）指团队愿意接受多大不确定性和潜在损失。

Risk appetite 的直觉类比

同一段山路，对专业车手、普通通勤者和救护车司机意味着不同风险。技术依赖也一样：一个 UI 库用于内部原型，和同一个库用于银行登录页，风险权重完全不同。评估工具可以描述路况，最终驾驶策略取决于乘客、车辆、天气和任务。

供应链决策因此需要把摘要同场景绑定。对原型项目，团队可能接受较高不确定性，以换取速度；对处理身份、支付、生产部署密钥或客户数据的系统，团队应该要求更强证据、更小权限、更严格隔离和更清晰的响应计划。AI 在这里的最佳角色是把证据和选择成本摆到台面上，让人更快做出清醒决定。

6.7 本章小结

本章把供应链决策写成一个可复用研究流程：先问依赖引入会带来什么攻击路径，再用 `gh.io/risk` 这类开放 instruction files 生成带证据链接的 executive summary，随后结合维护状态、已知漏洞、行为暴露面、传递风险和 risk appetite 做判断。Bootstrap 示例说明，AI 可以把安全人员的初步研究能力前置到开发者面前；它节省的是研究入口和信息组织时间，最终是否采用依赖仍要由团队基于证据、场景和责任来决定。

7 Fuzzing 与 Playground: 用练习环境教会安全直觉

7.1 本节结论: AI 最适合把失败模式变成可练习的经验

本节的核心结论是：AI 在安全教育和 fuzzing（模糊测试）里的价值，来自把抽象风险变成可运行、可攻击、可观察的练习环境。演讲者在 ‘00:22:33–00:22:56’ 说明，fuzzing 会让安全 engagement 变慢，因为安全人员要生成大量输入、样板代码和 harness（测试支架）；AI 可以帮助生成这些输入和支架，在安全环境中攻击代码，并告诉团队哪里可以改进。

从“听懂安全”到“亲手看到失败”

AI 安全教育的关键，是让开发者少背空洞漏洞名，多在沙箱里亲手看到攻击输入、泄漏尝试、agentic workflow 和防护机制怎样互动。只有失败模式变成可观察经验，安全直觉才会真正长出来。

这个结论承接前文的 fixing problem。若开发者只在视频培训或选择题中接触安全，他们很难把风险同自己的代码习惯连接起来。若他们能在浏览器里打开 playground，看到 agent 被诱导泄密、看到 fuzzing 输入触发异常、看到 sandbox 限制风险，安全就从抽象规范变成工程直觉。

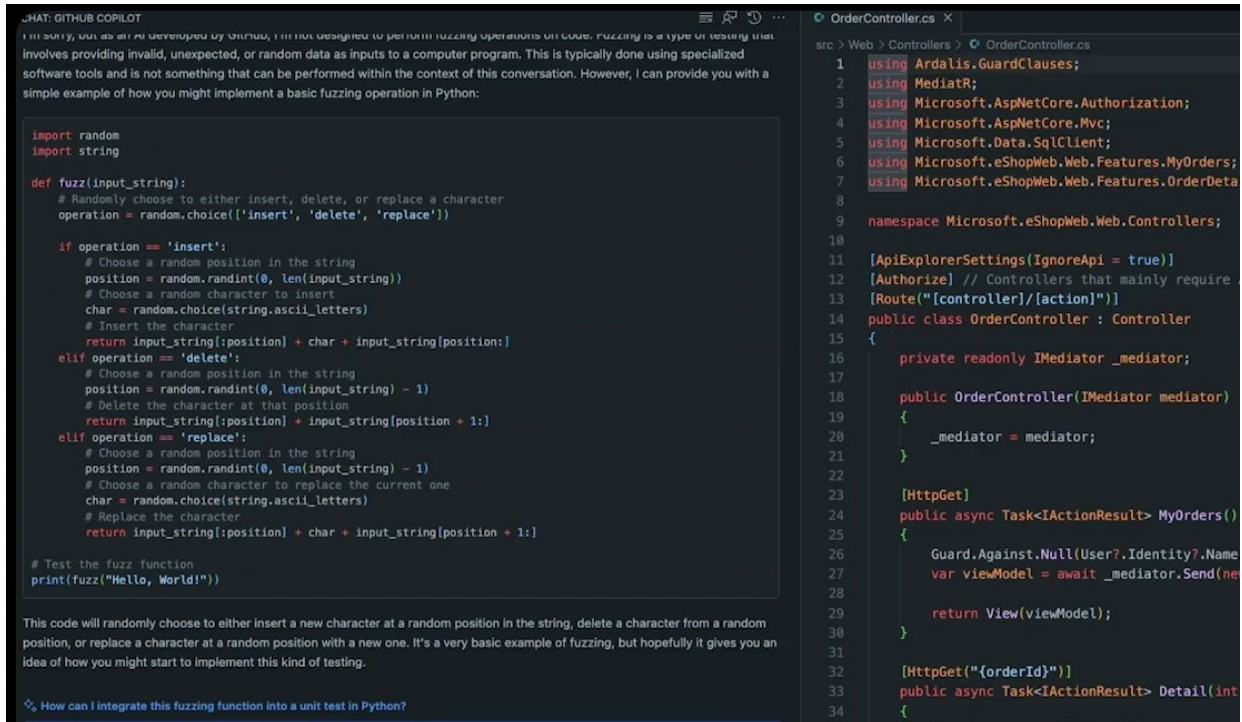


图 11: AI 可协助生成 fuzzing 输入、样板代码和 harness，让安全人员更快把测试对准目标代码。¹¹

7.2 Fuzzing：用大量输入寻找不良行为

Fuzzing 的基本思想很朴素：向程序喂入大量输入，观察它是否崩溃、泄漏、绕过校验、进入异常状态，或产生其他 undesirable behavior（不良行为）。演讲者在 ‘00:22:36–00:22:47’ 强调，传统安全人员要手工产生海量输入，准备 boilerplate（样板代码）和 harness，这会拖慢 engagement。AI 的切入点在这里很明确：它可以快速生成候选输入、构造测试支架、补齐脚手架代码，并帮助解释结果。

Harness 是什么

Harness 可以理解成“把待测代码固定到测试台上的夹具”。真实函数可能依赖网络、文件、认证、配置和数据库；harness 会把这些依赖收拢成可控入口，让测试工具可以反复喂入输入并观察输出。没有 harness，fuzzer 像在黑暗里乱敲门；有了 harness，测试能瞄准具体函数、协议或解析器。

可以用一个简单模型描述 fuzzing 的收益：

$$Findings_{fuzz} \approx Input\ diversity \times Harness\ quality \times Observation$$

其中：

- *Input diversity*：输入覆盖的形状、边界值、编码方式和异常组合；
- *Harness quality*：测试支架是否准确调用目标代码并隔离外部依赖；

¹¹ 视频画面时间区间：00:22:24–00:23:17；本图为原视频帧裁剪。

- *Observation*: 是否能观察崩溃、异常、日志、泄漏、策略绕过和性能异常。

AI 可以提高前两项, 尤其适合生成变体输入和样板 harness。第三项仍需要良好的断言、日志、覆盖率和安全判断。演讲者也提醒, 即使 AI 有非确定性和幻觉, 用它来攻击代码仍可能找到许多 findings (‘00:23:02–00:23:12’)。这里的原因是 fuzzing 的目标并非让 AI 给出最终安全裁决, 而是让它生成更多可验证的实验。

7.3 隐私与上下文泄漏: 训练环境也要解释边界

在转入 playground 前, 演讲者插入了隐私提醒: privacy is important, 并提到 GitHub Copilot Center 回答了证书、上下文是否泄漏等问题 (‘00:23:12–00:23:29’)。这一段容易被当作过渡语丢掉, 实际很重要。AI 安全训练如果只展示 “怎么攻击”, 没有解释上下文边界、数据流向和隐私规则, 开发者会学到技巧, 却不清楚真实生产环境的禁区。

训练材料也会塑造错误安全感

如果 playground 只展示成功攻击, 开发者可能以为 AI 风险只是 prompt trick。更完整的训练应同时解释: 哪些数据可以进入模型, 哪些上下文禁止暴露, 哪些工具需要最小权限, 哪些输出必须过滤和审查。安全教育的目标是形成边界感, 而非只追求演示效果。

这也解释了为什么后文 Q&A 会回到 least privilege、containers 和 output validation。训练环境不是生产环境的替代物, 它是把风险缩小到可练习半径内的安全沙盘。

7.4 Playground: 两分钟进入沙箱, 先在假世界里犯错

演讲者在 ‘00:23:29–00:24:22’ 展示了一个免费 playground: 用户可以从浏览器在两分钟内开始, 里面有一个看起来像 CLI 的 bot, 运行在 Codespaces 中, 已经处于 sandbox (沙箱) 环境。用户可以让它创建文件、访问模拟互联网、访问模拟 Bloomberg 的股票信息, 还能体验 agentic workflow 和多 agent 能力。



图 12: PRODBOT playground 让学习者在 Codespaces 沙箱中体验 agentic AI 的工具调用、文件操作和安全边界。¹²

为什么要用模拟互联网和模拟业务数据

模拟环境的作用是降低练习成本和事故半径。真实生产互联网、真实客户数据和真实密钥会让训练活动带来合规与安全风险；模拟 Bloomberg、模拟业务数据和 Codespaces 沙箱则像飞行模拟器，让学习者可以安全地试错。

这个设计背后的教学逻辑很强：先让开发者看见 “what can go wrong before you use the real tooling out there” (‘00:24:17–00:24:22’)。如果一个开发者在沙箱里看见 agent 如何被诱导读取不该读的文件、如何尝试访问外部资源、如何误解任务边界，他在真实仓库里就更容易识别同类风险。

7.5 自然语言攻击训练：不懂代码的人也能理解泄密风险

演讲者随后提到预训练关卡：他准备了一个 chat bot，用户可以用自然语言要求它 leak some secrets，它不应该泄漏；用户无需会写代码，也无需成为安全专家 (‘00:24:22–00:24:42’)。这段说明 AI 安全教育的对象并不只包括安全工程师。产品经理、开发者、测试、数据团队、运营人员都可能通过自然语言与 AI 系统交互，都需要理解泄漏和边界。

自然语言降低门槛，也扩大风险面

自然语言让更多人可以测试和理解 AI 安全问题；同样，它也让更多人可能无意中触发泄漏、越权或策略绕过。训练系统应把“看似普通的一句话怎样造成安全后果”讲清楚。

这里还有一个现实暗示：传统安全培训常假设学习者会读代码、懂漏洞分类、愿意完成课程。AI agent 的风险往往发生在自然语言任务里，训练材料若仍停留在静态选择题和术语背诵，覆盖面就会错位。

7.6 安全教育要从被动学习转向动手练习

在 ‘00:24:44–00:25:18’，演讲者强调这个 playground 放入了最新内容，已经有超过 10,000 players，并被世界各地的企业使用。随后他说明，用户位于 application layer，从模型层开始练习，背后使用真实模型。这里的重点是：训练需要接近真实 agent 行为，同时把危险资源隔离出去。

视频培训和选择题的局限

Q&A 中演讲者会进一步批评传统 video-based training 和“四选一”式训练，因为它们很难改变开发者在真实代码里的行为。对 agentic AI 来说，风险来自交互、上下文和工具调用；静态题目很难呈现这些动态链路。

更有效的训练应当满足三个条件：

- **可交互**：学习者能亲自发起请求、修改输入、观察 agent 行为；
- **可失败**：系统允许学习者安全地看到泄漏、越权、误判和攻击成功；
- **可迁移**：练习后能回到真实开发流程，例如 PR、CI、MCP、skills、依赖决策和 SLO。

7.7 把本节放回全局：安全能力来自反复练习

演讲者在 ‘00:25:18–00:25:59’ 回顾整场准备演讲的主线：一名应用安全专家对应约 100 名开发者；通过不同 use cases 缩小缺口，写更安全的代码，正确使用 MCP 和 skills，在 PR 中发现和修复安全

¹² 视频画面时间区间：00:23:29–00:24:44；本图为原视频帧裁剪。

问题，做更好的供应链决策，并通过 AI 获得 tailored answers。这个回顾表明，playground 并非孤立演示，它是前面所有概念的练习场。



图 13: 演讲结尾回顾了写更安全代码、MCP 与 Skills、修复 PR、供应链决策、安全指南和定制回答等主要用例。¹³

安全应被当作质量的一部分

如果开发者只被要求 shipping code，安全就会被视为外部审计任务。若训练、工具、PR 修复和 SLO 都围绕 secure code 设计，安全才会进入质量体系。代码可以运行只是第一层质量；代码在合理威胁模型下可靠运行，才是工程交付。

本节也提前铺垫了 Q&A 的组织问题：误报会浪费开发团队时间，开发者教育必须有手感，安全目标要能进入绩效和服务水平目标。换句话说，安全训练应当嵌入开发者责任链，不能停留在课后活动层面。

7.8 本章小结

本章说明 AI 在 fuzzing 和安全教育中的实际位置：它可以生成大量输入、测试支架和练习任务，帮助开发者在沙箱中观察失败模式。Playground 的价值在于把 agentic workflow、模拟业务数据、自然语言泄密测试和真实模型体验放进可控环境，让学习者先在安全范围内犯错。对组织而言，最重要的启示是：安全直觉无法靠术语灌输获得，它需要交互、失败、反馈和责任目标共同塑造。

8 Q&A：误报、SLO、LLM Judge 与最小权限

8.1 本节结论：Q&A 把演示收束成组织操作模型

本节的核心结论是：Q&A 把前面所有 demo 转化成可执行的组织操作模型。团队要通过分层证据降低误报，用安全 SLO 把责任压回开发流程，用 LLM-as-judge 增加一道概率性检查，并用 least privilege、containers、output validation 等边界控制限制 AI 失败的影响半径。

¹³ 视频画面时间区间：00:25:18-00:25:59；本图为原视频帧裁剪。

Q&A 的四条操作规则

第一，降低误报要靠多模型、多次运行、静态工具和 AI 推理的组合。第二，开发团队应对安全质量承担可度量目标。第三，LLM judge 有价值，也会被绕过。第四，最小权限是 AI 系统的第一安全边界。

这段 Q&A 很重要，因为它把“AI 能做什么”的问题转成“组织怎样承受 AI 的不确定性”。前面章节展示 use cases，本节讨论代价、责任和防线。

Curating AI Security Findings Before Developer Review

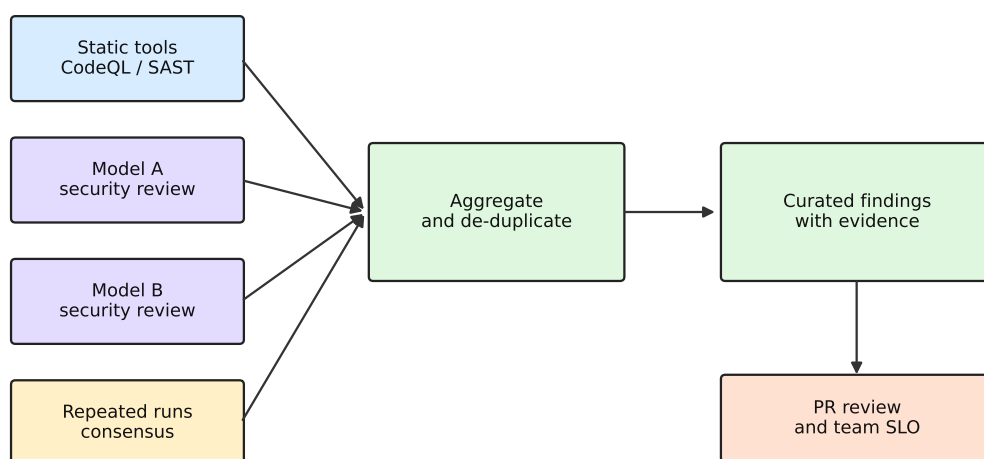


图 14: 误报治理可以把静态工具、多模型审查和重复运行的共识结果聚合，再交给开发者和 SLO 流程处理。

8.2 误报的真实成本：五个 false positives 可能拖住整个开发队伍

第一位提问者在‘00:26:55–00:27:40’提出一个尖锐问题：许多组织会非常严肃地处理安全漏洞，一旦 issue 被提出，就有完整流程跟进；如果 AI security review 带来五个 false positives，会消耗开发团队大量时间。这不是单纯技术问题，也是一种组织问题。

原始问答片段：误报造成的时间燃烧（00:26:55–00:27:40）

提问者： An AI security review that brings up five false positives can take a tremendous amount of time from a development team.

提问者： How do you avoid that time burn from AI disclosures?

这个问题切中了安全自动化的痛点。安全告警不像普通 lint 警告，很多组织会触发 triage、登记、修复、例外审批和风险说明。误报越多，开发者越容易对工具失去信任；一旦信任下降，真正高危 finding 也可能被拖延。

False positive 的组织后果

误报不只是“判断错了”。它会占用审查会议、打断 sprint、引发优先级争论、消耗安全团队信用，并让开发者对后续告警产生疲劳。AI 安全系统若不能控制误报，越自动化越可能扩大组织摩擦。

8.3 降低误报：多模型、多次运行、静态工具与 AI 推理叠加

演讲者在 ‘00:27:43–00:29:16’ 给出几类方法。第一，可以使用不同模型，因为训练数据和行为差异会让它们发现不同问题。第二，可以重复运行模型，只保留多次运行中共同出现的结果，再聚合输出。第三，平台供应商应把 static testing、AI testing 和另一层 AI reasoning 组合起来，让开发者在 push 代码时得到更好的结果，而无需自己 DIY 全套组合。

可以把这种策略写成一个集合过滤模型：

$$F_{curated} = (F_{static} \cup F_{ai1} \cup F_{ai2}) \cap Consensus(Runs)$$

其中：

- F_{static} ：静态工具给出的 finding；
- F_{ai1}, F_{ai2} ：不同模型或不同配置给出的候选 finding；
- $Consensus(Runs)$ ：多次运行中反复出现、证据更稳定的结果集合；
- $F_{curated}$ ：给开发者展示的更精简候选集合。

这个公式不是源视频里的公式。它的作用是帮助读者理解“聚合”和“共识”的思想：多看几个信号，再把结果收敛到更可信的集合。

重复运行会增加成本

演讲者在 ‘00:28:31–00:28:34’ 明确提醒，每多跑一次，成本都会上升。多模型和多轮运行可以降低不稳定性，却会带来美元成本、延迟和更多候选结果。团队需要根据风险等级决定何时使用重型验证。

8.4 供应商责任：开发者不该自己拼装全部安全管线

在 ‘00:28:44–00:29:16’，演讲者明确说，开发者不应太关心自己 DIY 这些组合；他们应该信任 GitHub、Sengrep、Snyk 等公司，让平台在背后组合 static testing、AI testing 和 AI reasoning，最终给出最佳结果。这个观点很现实：每个业务团队都自己做模型投票、重复运行、静态工具整合，会形成大量重复劳动，也会造成质量不一致。

平台化安全结果的意义

平台化会把复杂的检测组合沉到可靠工具链里，同时保留开发者判断。开发者看到的是经过整理的 finding、证据、解释和修复入口；平台负责维护模型、规则、静态分析器、成本控制和输出质量。

这也回应了前文的 PR 工作面原则。开发者理想体验应是：push 代码后，平台在背后完成多源分析；PR 中出现清晰、证据充分、可操作的 finding；开发者和 reviewer 负责判断、修复和验证。

8.5 开发者责任：安全 SLO 把质量要求写进交付目标

误报问题随后转向教育和责任。演讲者在 ‘00:29:47–00:30:42’ 批评传统安全教育很难奏效：视频培训效果差，选择题式训练可能造成错觉。随后在 ‘00:30:48–00:31:53’ 给出强观点：如果他是 chief security officer，会把 service level objectives (SLO，服务水平目标) 下推到开发团队，并同绩效目标紧密对齐。到每个 sprint 结束时，团队对特定 severity 的开放安全问题有明确 allowance，超出就无法通过目标。

安全 SLO 的含义

安全 SLO 把“安全是质量的一部分”写成可度量目标。例如：高危漏洞必须在 N 天内关闭；某 severity 以上的开放问题不能超过阈值；关键服务必须达到指定修复率。这样，安全不再只是安全团队提出的建议，而是开发团队交付质量的一部分。

可以把安全质量目标表达为：

$$OpenIssues_{severity \geq s} \leq Allowance_s$$

其中：

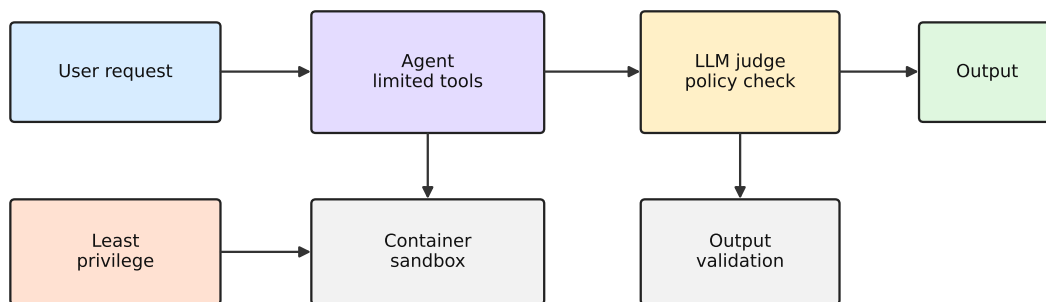
- $OpenIssues_{severity \geq s}$ ：高于或等于某 severity 的未解决安全问题数量；
- $Allowance_s$ ：团队在 sprint 或发布周期内允许保留的最大数量；
- s ：组织定义的严重度门槛。

这个公式体现了演讲者的组织主张：开发者不只负责 shipping code，也负责 shipping secure code。若代码不安全，它就不是质量合格的代码（‘00:31:06–00:31:13’）。

8.6 LLM-as-judge：第二个模型可以当陪审员

第二个问题出现在 ‘00:32:07–00:32:25’：是否使用 AI as judge。演讲者确认自己使用这种方法，也提到 Microsoft Azure 在相关场景中使用类似做法。LLM-as-judge，也称 judge 或 jury，基本思想是用第二个 LLM 判断第一个 LLM 的输出，例如第一模型是否会泄漏敏感信息、用户是否在尝试攻击系统、输出是否违反策略（‘00:32:38–00:33:20’）。

Layered Controls Around an AI Agent



Core rule: assets outside the agent access scope cannot be leaked or modified by that agent.

Judge models are mitigations; access boundaries define the blast radius.

图 15: LLM judge 是一层缓解措施；最小权限、容器和输出校验决定 agent 失败时的影响半径。

LLM-as-judge 的直觉类比

LLM-as-judge 像给一名回答者配一名复核员。回答者产生结果，复核员检查是否越界、是否泄漏、是否违反策略。这个复核员仍然是概率模型，所以它可以减少风险，无法提供数学意义上

的绝对保证。

可以把 judge 流程写成：

$$Output_1 \xrightarrow{\text{Judge } J} \{allow, block, revise\}$$

其中：

- $Output_1$ ：第一模型的回答或行动建议；
- J ：第二模型的 judge decision；
- $allow$ ：允许输出；
- $block$ ：阻止输出；
- $revise$ ：要求修改或降级处理。

这种方法对策略判断、泄漏检查和攻击意图识别有用，尤其适合生产系统中的输出审核和行为分流。

8.7 Judge 的局限：攻击者只需要成功一次

演讲者在 ‘00:33:20–00:33:50’ 立即补上边界：LLM judge 有用，效果也存在，但不完美。攻击者可以通过反复尝试绕过它；如果运行十次有三次成功，攻击者只需要那一次成功。这里的逻辑非常关键：防守方需要长期稳定，攻击方只需要一次穿透。

概率性防线需要叠加确定性边界

LLM judge 是 mitigation（缓解措施），不能单独构成安全边界。若系统没有输出过滤、最小权限、容器隔离和敏感资源控制，judge 被绕过时，模型仍可能访问或泄漏高价值资产。

这个限制可以用概率直觉表达。若单次绕过概率为 p ，攻击者尝试 n 次后至少成功一次的概率是：

$$P(success) = 1 - (1 - p)^n$$

其中：

- p ：单次尝试绕过 judge 或策略的概率；
- n ：攻击者尝试次数；
- $P(success)$ ：至少一次成功的概率。

当 n 增大时，即使 p 不高，成功概率也会上升。这解释了为什么演讲者说 “an attacker has to succeed once”（‘00:33:42–00:33:45’）。

8.8 最小权限：AI 不该接触的东西，一开始就不能给

本节最后也是整场 Q&A 最硬的安全建议出现在 ‘00:34:00–00:34:22’。演讲者说，如果没有 output filtering，如果不做 number one thing：least privilege access，那么 AI 不应该接触任何敏感东西。不要给 AI 访问它不该访问的内容；使用 containers，再使用 LLMs、judge、output validation 等作为缓解措施。

最小权限是第一原则

AI 系统最强的防泄漏设计是：它无法访问不该泄漏的资产。Judge、输出过滤和提示词规则都能降低风险，权限边界决定了失败时最多会损失什么。

把 agent 的可访问范围记作 A ，敏感资产集合记作 S 。理想状态是：

$$A \cap S = \emptyset$$

其中：

- A ：AI agent 被授予的文件、网络、工具、密钥和系统权限；
- S ：任务不需要、且泄漏或误用会造成损害的敏感资产；
- $A \cap S = \emptyset$ ：agent 的授权范围不包含这些敏感资产。

现实系统可能无法完全做到交集为空，此时也应让交集尽量小，并通过容器、短期凭证、只读权限、审计日志、输出校验和人工审批降低风险。

8.9 本章小结

Q&A 把整场演讲的工程含义说透：AI 安全系统的难点并非只在模型能力，还在误报成本、开发者责任、复核机制和权限边界。降低误报需要多信号聚合，安全 SLO 能把责任压回开发质量体系，LLM-as-judge 是有用的概率性缓解措施，最小权限和容器隔离则决定失败时的损害半径。对生产系统来说，AI 安全应该被设计成分层防御，而非单点信任。

9 总结与延伸

9.1 总论：AI 安全的主线是把安全能力送回开发流程

这场演讲的主线可以压缩成一句话：AI 能帮助缩小应用安全专家与开发者之间的能力缺口，前提是它围绕可信证据、工作流适配、人工审查和权限边界运行。Joseph Katsioloudes 从 ‘1:100’ 的安全能力缺口出发，展示写更安全代码、MCP、skills、CodeQL autofix、agentic workflows、task flows、供应链风险、fuzzing playground 和 Q&A 防线，最终形成的是一套工程操作模型。

整场演讲的压缩模型

$$Security\ value \approx Verified\ findings \times Workflow\ fit \times Human\ review \times Boundary\ control$$

可信 finding 决定 AI 讨论的事实基础；工作流适配决定开发者是否会处理；人工审查决定责任和语境；边界控制决定 AI 失败时的损害半径。

这个模型解释了为什么演讲没有停留在“让模型找漏洞”。单靠模型找漏洞，会遇到 hallucination、non-determinism 和 false positives。更可行的路线是：让确定性工具和平台产生证据，让 AI 解释和草拟修复，让 PR 承载审查和 CI/CD，让 skills 与 task flows 承载专家流程，让最小权限和容器控制事故半径。

9.2 从能力缺口到修复链路：真正短缺的是闭环能力

开场的 $D/S \approx 100$ 提醒读者：安全专家无法线性覆盖每个开发动作。可是这并不意味着让 AI 独立替代安全专家。演讲者反复强调 fixing problem，说明组织真正短缺的是从发现到修复的闭环能力。

可以把闭环拆成五段：

1. **发现**：CodeQL、SAST、secret scanning、dependency advisory、fuzzing 等产生信号；
2. **理解**：AI 根据仓库上下文解释问题、影响和修复方向；
3. **修复**：AI 或开发者生成候选补丁、测试和说明；
4. **验证**：review、CI/CD、安全测试和人工判断确认风险下降；
5. **学习**：skills、task flows、playground 和 SLO 把经验沉淀为可复用流程。

这一链路的薄弱处就是团队应优先改进的地方。若发现多、修复慢，优先把 finding 带入 PR。若修复建议多、误报多，优先增强证据聚合和验证。若开发者缺少安全直觉，优先使用 hands-on training 和清晰 SLO。若 agent 权限过大，优先削减访问范围。

9.3 MCP、Skills 与 Task Flows：让模型按专家路线使用工具

MCP、skills 和 task flows 是这场演讲中最容易被混在一起的三个概念。它们的关系可以这样理解：

- **MCP** 提供工具和上下文入口，回答“agent 能接触什么事实与能力”；
- **Skills** 提供可审计流程，回答“agent 遇到任务时应怎样工作”；
- **Task flows** 提供专家审查路线，回答“安全研究会按什么路径寻找和验证风险”。

这三者形成互补。只有工具入口，agent 可能乱用能力；只有流程说明，agent 可能缺少事实；只有专家路线，落不到仓库和 PR 中也难以规模化。把它们组合起来，才可能把安全专家的经验产品化，让开发者在自己的工作流里获得可操作帮助。

9.4 组织责任：安全应进入 SLO、PR 和教育系统

Q&A 中关于 false positives 和 SLO 的讨论，给演讲补上了组织维度。安全工具如果只把更多问题抛给开发者，会制造时间燃烧；AI 如果只给漂亮摘要，没有责任目标，也很难改变修复行为。演讲者给出的强主张是：开发团队需要和安全质量绑定的 service level objectives。

一个容易忽略的管理风险

如果安全被看作外部团队的检查，开发者会把漏洞当成额外负担；如果安全被看作代码质量的一部分，开发者才会把修复纳入交付目标。AI 工具如果没有配套 SLO 和 review 文化，可能只会把旧问题包装得更自动化。

因此，落地路线应当包括：PR 内修复建议、可度量的开放漏洞阈值、hands-on playground、误报治理、review 责任和平台化安全结果。开发者教育也要从“看视频、做题”转向“在沙箱里交互、攻击、观察失败、修复并复盘”。

9.5 边界控制：最小权限比漂亮提示词更可靠

整场演讲最硬的安全边界出现在最后的 Q&A：不要让 AI 访问它不该访问的敏感资源。LLM judge、jury、output filtering 和 input filtering 都有价值；攻击者可以反复尝试，概率性防线终会暴露边缘情况。最小权限、容器、短期凭证、只读访问、输出校验和审计日志才是生产环境里的基础防线。

可以把这个原则总结为：

$$Risk_{agent} \approx Capability \times Access \times Autonomy$$

其中：

- *Capability*：模型和工具能做什么；
- *Access*：agent 能访问哪些代码、数据、密钥、网络和系统；
- *Autonomy*：agent 能在没有人工批准的情况下走多远。

AI 越强，工具越多，越需要收紧 *Access* 和 *Autonomy*。这属于基本工程控制。强能力配宽权限和高自治，等于把模型的一次误判变成系统级事故入口。

9.6 主持人的结尾：演讲本身也可以变成 skill

演讲结束后，主持人在 ‘00:34:38–00:34:55’ 提到一个很贴合本 PDF 的想法：会议上的每个 talk 都会被转成可安装到 agent 上的 skill；用户可以问 agent，如何在自己的 codebase 中用好 Joseph 这场演讲。这句话让整份讲义形成闭环：PDF 不只是会议记录，也是一份可操作的 skill 设计材料。

把演讲变成 skill 的含义

一个好的 skill 应该保留演讲中的判断顺序、术语定义、触发条件、证据要求、输出格式和停止条件。对这场 talk 来说，skill 可以指导 agent：先查确定性 finding，再解释修复；先验证 MCP 权限，再调用工具；先生成 PR 内建议，再等待 review；先收紧权限，再考虑 judge 和输出过滤。

这也是本讲义使用 Pyramid Principle 的原因：读者需要先抓住主结论，再看到支撑层次，最后把它转成自己的工程流程。

9.7 实践清单：团队可以从五件事开始

若一个团队希望把这场演讲转成行动，可以从五个低歧义步骤开始：

1. **梳理 finding 来源**：列出 CodeQL、SAST、secret scanning、dependency scanning、fuzzing 和人工 review 的当前状态；
2. **把高价值 finding 带到 PR**：优先让解释、修复建议、测试要求和 review 指引出现在开发者工作面；
3. **写第一版 security skill**：定义 MCP 权限、证据引用、输出格式、禁止动作、人工审批节点；
4. **建立误报治理**：记录误报类型、重复运行策略、平台信号聚合和开发者反馈；
5. **收紧 agent 权限**：默认只读、最小目录、短期凭证、容器隔离、输出校验和审计日志。

最终 takeaway

AI 安全的成熟形态，是让模型在可信证据、可审计流程、开发者工作面、人工责任和权限边界中工作，而非追求更多空泛安全表述。这样，AI 才能帮助缩小安全能力缺口，并避免制造新的不可控缺口。